

# When Are Text/LLM Embeddings Valid for Causal Adjustment?

Embedding-Based Causal Adjustment in PSID

## **Team Members:**

Ishaan Agarwal (ia299)

Janice Yu (jy2238)

Jingze Dai (jd2379)

Jiyao Xi (jx449)

Xingkun Xie (xx377)

## **Project Advisor:**

Professor Jelena Bradic

May 11, 2026

# Executive Summary

This project investigates whether text-based and LLM-style embeddings can serve as valid adjustment variables for causal effect estimation in observational panel data. The empirical application focuses on the effect of job-related training on later household income using the Panel Study of Income Dynamics (PSID). Since job training is not randomly assigned, treated and untreated households may differ systematically in pre-treatment income, education, employment history, assets, health, family structure, and other latent factors such as motivation or labor market attachment. Therefore, a central challenge is whether richer pre-treatment representations can improve causal adjustment beyond traditional structured covariates.

We construct a household-level PSID panel by aligning family-level records through a stable household-head PID. Treatment is defined using job-related training participation in the 2013 and 2015 waves, while outcomes are measured using post-treatment total family income from 2019 and 2021. All adjustment information is restricted to the pre-treatment period. In addition to structured covariates  $X$ , we generate deterministic pseudo-text notes  $W$  from pre-treatment household histories. These notes summarize demographic, economic, housing, employment, income, asset, program participation, and health information using a fixed schema with explicit missing-value encoding, ensuring reproducibility and avoiding post-treatment leakage.

The main methodological contribution is the construction and evaluation of multiple embedding pipelines that transform pseudo-text  $W$  into numerical representations  $R = \phi(W)$ . These include sparse TF-IDF/SVD representations, transformer-based sentence embeddings, trajectory embeddings, multi-view fusion pipelines, overlap-aware projections, contrastive stability-tuned embeddings, and stacked ensemble embeddings with fold-aware PCA. These representations are evaluated under three adjustment regimes: covariates only ( $X$ ), embeddings only ( $R$ ), and fused features  $[X; R]$ .

To evaluate causal performance objectively, we use a semi-synthetic benchmark built on real PSID pre-treatment data. Treatment assignment and potential outcomes are generated

from a known data-generating process that includes both observed covariates and a latent confounder. This design allows the true average treatment effect to be computed directly, so competing estimators can be compared using bias, RMSE, confidence interval coverage, overlap diagnostics, and robustness across difficulty tiers. The causal estimators include difference in means, regression adjustment, inverse probability weighting, Hájek IPW, augmented inverse probability weighting (AIPW), and double machine learning.

Overall, the results show that embedding-based adjustment can substantially improve point estimation accuracy in the semi-synthetic benchmark. Across all four difficulty tiers, the best  $R$ -only representations achieve lower RMSE than the  $X$ -only baseline, suggesting that text embeddings can summarize useful confounding information from household histories. Low-dimensional PCA embeddings, especially with  $k = 50$ , perform most reliably, while very high-dimensional embeddings often degrade both accuracy and coverage. The results also reveal an important trade-off:  $R$ -only adjustment generally provides the strongest point accuracy, whereas the fused  $[X; R]$  regime can improve confidence interval coverage in some settings. Among estimators, AIPW with random forest nuisance models is the most robust across pipelines and benchmark designs.

In the real PSID application, estimated average treatment effects of job-related training on later household income are statistically indistinguishable from zero across adjustment regimes and estimators. This should not be interpreted as proof of no true effect, since the real data do not reveal the counterfactual outcomes. Rather, the analysis indicates that after adjustment for observed and learned pre-treatment representations, there is no statistically significant evidence of an effect in this sample. Taken together, the project shows that embedding-based representations are promising tools for causal adjustment, but they must be used with careful dimensionality reduction, overlap diagnostics, and estimator choice.

# Contents

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction and Research Questions</b>           | <b>6</b>  |
| 1.1      | Background . . . . .                                 | 6         |
| 1.2      | Motivation . . . . .                                 | 7         |
| 1.3      | Research Questions . . . . .                         | 8         |
| <b>2</b> | <b>Data Description</b>                              | <b>8</b>  |
| 2.1      | PSID Panel Structure . . . . .                       | 8         |
| 2.2      | Treatment and Outcome Definition . . . . .           | 9         |
| 2.3      | Covariates $X$ . . . . .                             | 10        |
| 2.4      | Pseudo-text Construction $W$ . . . . .               | 11        |
| 2.5      | Target Estimand . . . . .                            | 13        |
| <b>3</b> | <b>Methodology Overview</b>                          | <b>14</b> |
| 3.1      | Overall Framework . . . . .                          | 14        |
| 3.2      | Representation Learning Strategy . . . . .           | 15        |
| 3.3      | Semi-synthetic Benchmark Design . . . . .            | 16        |
| 3.4      | Evaluation Strategy . . . . .                        | 16        |
| <b>4</b> | <b>Embedding Pipelines</b>                           | <b>17</b> |
| 4.1      | Pipeline A . . . . .                                 | 17        |
| 4.2      | Pipeline B . . . . .                                 | 18        |
| 4.3      | Pipeline C . . . . .                                 | 19        |
| 4.4      | Pipeline D . . . . .                                 | 20        |
| 4.4.1    | Multi-View Construction and Text Embedding . . . . . | 21        |
| 4.4.2    | Pipeline D1: Concatenation + PCA . . . . .           | 22        |
| 4.4.3    | Pipeline D2: Learned Linear Fusion . . . . .         | 23        |
| 4.4.4    | Pipeline D3: Attention-based Fusion . . . . .        | 25        |

|          |                                                                                                  |           |
|----------|--------------------------------------------------------------------------------------------------|-----------|
| 4.5      | Pipeline E . . . . .                                                                             | 26        |
| 4.6      | Pipeline F . . . . .                                                                             | 30        |
| 4.7      | Pipeline G . . . . .                                                                             | 34        |
| <b>5</b> | <b>Semi-synthetic Benchmark Design</b>                                                           | <b>36</b> |
| 5.1      | Framework Overview . . . . .                                                                     | 36        |
| 5.2      | Latent Confounding and Treatment Assignment . . . . .                                            | 37        |
| 5.3      | Potential Outcome Models . . . . .                                                               | 38        |
| 5.4      | Difficulty Tiers and Treatment Heterogeneity . . . . .                                           | 39        |
| 5.5      | Ground-truth Effects and Evaluation . . . . .                                                    | 40        |
| <b>6</b> | <b>Causal Estimation Methods</b>                                                                 | <b>41</b> |
| 6.1      | Baseline Estimators . . . . .                                                                    | 41        |
| 6.1.1    | Difference in Means . . . . .                                                                    | 41        |
| 6.1.2    | Regression Adjustment . . . . .                                                                  | 42        |
| 6.1.3    | Inverse Probability Weighting (IPW) . . . . .                                                    | 44        |
| 6.2      | Doubly Robust Estimators . . . . .                                                               | 47        |
| 6.2.1    | Augmented Inverse Probability Weighting (AIPW) . . . . .                                         | 47        |
| 6.2.2    | Double Machine Learning (DML) . . . . .                                                          | 49        |
| <b>7</b> | <b>Results</b>                                                                                   | <b>51</b> |
| 7.1      | Semi-Synthetic Monte Carlo Benchmarks . . . . .                                                  | 51        |
| 7.1.1    | Embeddings Consistently Outperform Covariates on Point Accuracy . . . . .                        | 51        |
| 7.1.2    | Low-Dimensional PCA Embeddings ( $k = 50$ ) Dominate; High Dimensions Are Catastrophic . . . . . | 54        |
| 7.1.3    | Concatenating Covariates Degrades Point Accuracy but Improves Coverage . . . . .                 | 56        |
| 7.1.4    | AIPW with Random Forest Nuisance Models Is the Most Robust Estimator . . . . .                   | 59        |

|       |                                                       |    |
|-------|-------------------------------------------------------|----|
| 7.1.5 | Practical Recommendations from the Study . . . . .    | 60 |
| 7.2   | Analysis of the PSID Panel Data . . . . .             | 61 |
| 7.2.1 | Dataset Description and Experimental Design . . . . . | 61 |
| 7.2.2 | Results . . . . .                                     | 61 |
| 8     | Conclusions and Recommendations                       | 62 |
| 9     | Limitations and Future Work                           | 64 |
| 10    | Acknowledgments                                       | 65 |
|       | Acknowledgments                                       | 65 |
|       | References                                            | 67 |

# 1 Introduction and Research Questions

## 1.1 Background

Causal effect estimation in observational data is fundamentally more difficult than prediction because treatment assignment is not randomized. In this project, we study the effect of job-related training on later household earnings using PSID panel data. If households that receive training differ systematically from those that do not, then a simple comparison of observed outcomes will generally mix the effect of training with pre-existing differences between the two groups. As a result, credible causal estimation requires adjustment for variables that affect both treatment assignment and the outcome.

A standard approach in causal inference is to adjust for observed pre-treatment covariates such as demographic, economic, and household characteristics. Under a conditional ignorability assumption, treatment can be viewed as independent of the potential outcomes after conditioning on a sufficiently informative adjustment set. In practice, however, this assumption is often difficult to justify in social and economic data. Important factors such as motivation, earnings potential, labor market attachment, and other household-level constraints may be only partially observed or not directly measured at all. When such factors influence both participation in training and later income, adjustment based only on a structured covariate set may still leave substantial confounding bias.

This challenge motivates the use of richer pre-treatment representations. In our project, household histories are summarized as pre-treatment pseudo-text and then mapped into learned representations through embedding pipelines. The central idea is that text-style summaries may preserve information about household circumstances, work history, and prior economic conditions that is not fully captured by a conventional tabular design matrix. If these learned representations retain information relevant to both treatment and outcome, they may serve as useful adjustment variables for causal estimation. At the same time, richer representations are not automatically beneficial: they must reduce confounding without

destroying overlap between treated and untreated units. This trade-off is the main background problem that motivates the rest of the report.

## 1.2 Motivation

This project is motivated by the practical challenge of estimating causal effects in longitudinal socioeconomic data such as PSID. In this setting, participation in job-related training is not randomly assigned. Instead, it is influenced by a range of household characteristics, including prior income, employment history, education, and family conditions. As a result, treated and untreated households are systematically different, making it difficult to isolate the effect of training on future earnings.

Although PSID provides a rich set of structured covariates, these variables may not fully capture all relevant factors that influence both treatment and outcome. Important characteristics such as motivation, long-term earning potential, and labor market attachment are often only partially observed or imperfectly measured. Even with expanded covariate sets that include income history, health status, and asset information, there may still remain residual confounding due to these latent factors.

This motivates the exploration of alternative representations that can better summarize pre-treatment information. In this project, we construct pseudo-text descriptions of household histories and use embedding methods to transform them into low-dimensional representations. The goal is to capture complex patterns in household trajectories that may not be fully represented by individual structured variables. However, it is not clear whether such representations can serve as valid adjustment variables in causal estimation. They may help reduce confounding by encoding richer information, but they may also introduce challenges such as loss of relevant information or reduced overlap between treated and untreated groups. Understanding this trade-off is the central motivation of this project.



## 1.3 Research Questions

Based on the challenges outlined above, this project aims to answer the following research questions:

- Can embedding-based representations serve as valid adjustment variables for causal effect estimation?
- How do different embedding constructions affect estimation accuracy, in terms of bias and RMSE, compared to traditional covariate-based adjustment?
- Do embedding-based representations preserve sufficient overlap between treated and untreated groups to ensure stable estimation?
- How robust are embedding-based adjustments to perturbations in pseudo-text construction and variations in modeling choices?

## 2 Data Description

### 2.1 PSID Panel Structure

The data used in this project are drawn from the Panel Study of Income Dynamics (PSID), a longitudinal household survey that has followed U.S. families and individuals over multiple decades. The PSID is designed as a panel dataset in which households are repeatedly observed over time, allowing for the study of economic dynamics across years and generations.

A key challenge in constructing a multi-year panel from PSID arises from the fact that the family interview number changes across waves. As a result, family-level records cannot be directly linked over time using the reported family identifiers. To address this issue, we construct a stable individual-level identifier (PID) by concatenating the original PSID person identifiers. This PID remains constant across waves and serves as the primary key for all cross-year merging.

Because family-level files do not directly contain this stable identifier, we link family-level observations to individuals through the head of household in each wave. Specifically, the head is identified using the relation-to-head and sequence number information in the individual-level data. We then create mapping tables that associate each family interview number with the corresponding PID of the household head. Using these mappings, all family-level variables are reassigned to a stable PID and aligned consistently across time.

The resulting dataset is organized at the household level, with each observation corresponding to a household identified through the head’s PID. This structure ensures that all variables are consistently aligned across waves, providing a reliable foundation for defining treatment, outcomes, and pre-treatment covariates in subsequent analysis.

## 2.2 Treatment and Outcome Definition

The treatment variable in this study is defined based on participation in job-related training programs. Information on training is obtained from multiple survey questions that capture whether the head of household or the spouse has attended job training activities, as well as the type of institutions or organizations providing such training.

To construct a robust treatment indicator, we combine multiple training-related variables from two survey waves, 2013 and 2015. Within each wave, a strict rule is applied: participation is identified only when specific core indicators confirm training activity and the reported training types fall within a predefined set of valid categories. The variables within each year are aggregated using logical OR, and the final treatment indicator is defined as one if the individual satisfies the training criteria in at least one of the two years. This multi-year construction helps capture sustained or repeated engagement in training and reduces sensitivity to measurement noise in any single wave.

The outcome variable is defined as total family income observed after treatment. Specifically, we use post-treatment income measures from the 2019 and 2021 waves. These variables represent total family money income, which aggregates multiple sources of earnings and

may include both positive income and negative values indicating net losses. To ensure meaningful outcome information, an observation is considered valid if at least one of the two post-treatment income measures is non-missing and non-zero.

This temporal structure ensures a clear causal ordering: baseline covariates are measured prior to treatment, treatment occurs in 2013 and 2015, and outcomes are observed in later waves. This separation supports the interpretation of the estimated effects as the impact of job training on subsequent household income.

## 2.3 Covariates $X$

The covariate set  $X$  consists of a rich collection of pre-treatment household characteristics constructed from the PSID data. All covariates are measured prior to treatment to ensure a valid temporal ordering for causal adjustment. The goal of these variables is to capture key aspects of household socioeconomic status, demographic structure, and historical conditions that may jointly influence both participation in job training and subsequent income outcomes.

To provide a structured representation, the covariates are organized into several broad categories, including demographic characteristics, household composition, economic status, housing and asset information, employment and work history, and health-related measures. These categories summarize different dimensions of pre-treatment information and together form a comprehensive adjustment set.

Rather than relying on individual variables alone, this grouped structure allows the model to capture multiple aspects of household conditions, including long-term economic resources, labor market attachment, and health status. However, despite the richness of this covariate set, some relevant factors may still be imperfectly measured or unobserved, which motivates the use of additional representations introduced later in the analysis.

**Table 1:** Summary of Pre-treatment Covariates

| Category                    | Description                                                                                                                                                  |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Demographics                | Age, sex, race, and education level of the household head                                                                                                    |
| Household Composition       | Family size, number of children, and marital status                                                                                                          |
| Economic Status             | Labor income, total family income, and indicators of financial support (e.g., WIC participation)                                                             |
| Housing and Assets          | Home value, mortgage status, and ownership of financial assets such as savings, stocks, and retirement accounts                                              |
| Employment and Work History | Current employment status, years worked since age 18, and business ownership                                                                                 |
| Health and Behavior         | Physical and mental health indicators, including chronic conditions, psychological distress, and lifestyle behaviors such as smoking and alcohol consumption |

## 2.4 Pseudo-text Construction $W$

To convert structured pre-treatment covariates into a text-based representation suitable for embedding, we construct a deterministic pseudo-note for each household, denoted by  $W_i$ . The purpose of  $W_i$  is to provide a stable textual summary of household history that preserves pre-treatment information while remaining fully reproducible.

Formally, let  $X_i^{\text{pre}}$  denote the collection of all structured covariates for household  $i$  measured prior to treatment. We define the pseudo-text generator as a deterministic mapping

$$W_i = g(X_i^{\text{pre}}),$$

where  $g(\cdot)$  converts each record into a fixed-schema textual representation. In this study, the treatment year is set to 2013, so only information from 2007, 2009, and 2011 is included in  $W_i$ . This ensures that the constructed pseudo-notes strictly satisfy the no-leakage requirement for causal analysis.

Each household record is first aligned at the PID level, and all available pre-treatment variables are collected into a unified feature set. This includes demographic characteristics,

household composition, housing and asset information, work history, income history, program participation, health measures, and selected earlier-wave variables. These inputs together form the complete information base from which the pseudo-text is generated.

The construction of  $W_i$  proceeds by translating each structured variable into a factual sentence using fixed, deterministic rules. Binary variables are mapped to explicit yes/no statements, categorical variables are converted into stable descriptive labels, and numeric variables are rendered using consistent formatting conventions. This process produces a list of factual statements that fully describe the pre-treatment status of each household without introducing stylistic variation.

A key design choice is the explicit representation of missing information. Rather than omitting variables with missing values, each such field is retained in the pseudo-note and assigned the token `[MISSING]`. This ensures that all pseudo-notes share the same structural template and prevents variation in text length or content due solely to missingness. In contrast, structurally inapplicable values are encoded as explicit “not applicable” statements, preserving the original survey semantics and avoiding confusion between missing data and true inapplicability.

To further improve stability, selected variables are transformed using fixed recoding rules prior to text generation. Monetary variables are rounded to the nearest \$1,000, ages are grouped into 5-year intervals, and certain derived variables are expressed using coarse but interpretable categories. For example, the K6 psychological distress score is encoded using both a categorical severity label and a threshold indicator. These transformations reduce unnecessary numerical variability while maintaining meaningful pre-treatment information.

Finally, all factual statements are assembled into a pseudo-note using a fixed schema and ordering. Each  $W_i$  follows the same sequence of sections, including demographics, household characteristics, housing, work history, income history, assets, program participation, health status, and earlier-wave history. Within each section, variables appear in a consistent order. This fixed structure reduces order-induced variation and ensures that the resulting text

representation is stable across observations and across repeated runs.

The resulting pseudo-note  $W_i$  is therefore a deterministic, fully pre-treatment textual representation of household history. It preserves all available structured information in a consistent format, explicitly encodes missingness, and provides a suitable input for downstream embedding pipelines while maintaining transparency and reproducibility.

**Table 2:** Encoding Rules for Pseudo-text Construction

| Component             | Rule                                                                                         |
|-----------------------|----------------------------------------------------------------------------------------------|
| Pre-treatment window  | Only variables from 2007, 2009, and 2011 are used to construct $W_i$                         |
| Binary variables      | Converted into explicit yes/no factual statements                                            |
| Categorical variables | Converted into fixed descriptive labels using deterministic mappings                         |
| Numeric variables     | Rendered using stable formatting; selected variables are rounded or binned                   |
| Missing values        | Explicitly encoded as [MISSING] rather than omitted                                          |
| Inapplicable values   | Encoded as explicit “not applicable” statements rather than missing                          |
| K6 distress score     | Encoded using two lines: bucketed distress category and threshold indicator ( $K6 \geq 13$ ) |
| Section order         | Fixed schema applied to every household in the same order                                    |
| Output type           | Deterministic template pseudo-note with no free-form inference                               |

## 2.5 Target Estimand

The primary objective of this study is to estimate the causal effect of job-related training on subsequent household income. We focus on the average treatment effect (ATE), defined as the expected difference in potential outcomes between treated and untreated households at a post-treatment horizon  $h$ :

$$\text{ATE}_h = \mathbb{E}[Y_i^1(t+h) - Y_i^0(t+h)],$$

where  $Y_i^1(t+h)$  and  $Y_i^0(t+h)$  denote the potential outcomes under treatment and control, respectively.

In standard observational settings, the ATE is not directly observable and must be estimated under identification assumptions using observed data. In contrast, this study adopts a semi-synthetic framework in which the data-generating process is partially controlled. In particular, treatment and outcomes are generated based on both observed pre-treatment covariates and a latent confounder that is not available to estimation procedures.

Under this setup, both potential outcomes are defined by construction, which allows the ground-truth causal effect to be computed directly. This provides a benchmark for evaluating the accuracy of different estimation strategies, rather than relying solely on assumptions about identification.

The next section introduces the methodological framework used to construct this semi-synthetic setting, including the generation of latent confounding, treatment assignment, and outcome models.

## 3 Methodology Overview

### 3.1 Overall Framework

The objective of this study is to evaluate whether embedding-based representations can function as effective adjustment variables for causal inference. To answer this question, we design an integrated workflow that combines representation learning with controlled causal evaluation. The overall framework begins with the construction of a panel-aligned PSID household dataset containing pre-treatment covariates, treatment indicators, and post-treatment outcomes. From these structured variables, we generate pseudo-text summaries that encode household history and socioeconomic characteristics in a text-based format.

These pseudo-text inputs are then transformed into numerical representations through multiple embedding pipelines. The resulting embeddings are treated as alternative adjust-

ment variables and compared against traditional structured covariates. To enable objective evaluation, we embed these representations within a semi-synthetic benchmark framework in which treatment assignment and potential outcomes are generated under a known data-generating process. This setup provides access to ground-truth causal effects and allows direct comparison of competing estimation strategies.

The complete workflow can be summarized as:

PSID Data  $\rightarrow (X, W) \rightarrow R \rightarrow$  Semi-synthetic Benchmark  $\rightarrow$  Estimation  $\rightarrow$  Evaluation.

This modular design separates data construction, representation learning, causal estimation, and benchmarking, making it possible to assess the specific contribution of embeddings to causal adjustment.

### 3.2 Representation Learning Strategy

A central component of the project is the transformation of pre-treatment household information into representations that may better capture latent confounding structure. We consider two sources of adjustment information. The first is the conventional structured covariate set  $X$ , which includes demographic, economic, household, and health-related variables. The second is the pseudo-text representation  $W$ , which converts the same pre-treatment information into a deterministic textual summary.

Embedding methods are then applied to  $W$  (and in some settings jointly with  $X$ ) to produce low-dimensional numerical representations  $R$ . These representations are intended to summarize complex interactions, historical patterns, and latent socioeconomic characteristics that may not be fully captured by tabular variables alone. Depending on the pipeline, embeddings may be generated using sparse text methods, transformer-based sentence encoders, or multi-view fusion approaches.

The key empirical question is whether these learned representations preserve information



relevant to both treatment assignment and outcomes. If successful, embeddings may provide a richer adjustment set than conventional covariates. If unsuccessful, they may omit relevant confounding information or weaken overlap between treated and untreated groups. The next chapter provides detailed descriptions of the embedding pipelines used in the study.

### 3.3 Semi-synthetic Benchmark Design

To rigorously evaluate causal adjustment methods, we use a semi-synthetic benchmark rather than relying solely on observational comparisons. In this framework, realistic baseline covariates are retained from the original PSID data, while treatment assignment and outcomes are generated through a controlled data-generating process (DGP).

A latent confounder  $U$  is constructed from pre-treatment household history to represent unobserved factors such as motivation, earnings capacity, or labor market attachment. This variable affects both treatment assignment and outcomes but is hidden from estimation procedures. Treatment is then assigned probabilistically as a function of observed covariates and the latent confounder, while potential outcomes are generated under flexible models that allow nonlinear baseline effects and heterogeneous treatment effects.

Because both treatment assignment and potential outcomes are known by construction, the true causal effect can be computed directly. This feature makes it possible to evaluate estimation methods using objective criteria such as bias and root mean squared error. Detailed specifications of the benchmark DGP are presented in a later section.

### 3.4 Evaluation Strategy

The final component of the methodology is the comparison of alternative adjustment strategies under the common benchmark framework. We compare estimators that rely on structured covariates, embedding-based representations, or combinations of both. Standard causal estimators such as regression adjustment, propensity score weighting, and doubly robust methods are considered.

Performance is evaluated along several dimensions. First, estimation accuracy is assessed by comparing estimated treatment effects with the known ground truth. Second, overlap diagnostics are used to determine whether treated and untreated units remain comparable after representation learning. Third, robustness analyses examine sensitivity to modeling choices, random seeds, and alternative benchmark settings.

This evaluation strategy allows us to study not only whether embeddings improve predictive representations, but whether they improve causal adjustment in a measurable and statistically meaningful sense.

## 4 Embedding Pipelines

### 4.1 Pipeline A

Pipeline A is the baseline method transforming the combinations of covariates  $X$  and pseudo-notes  $W$  into vector-based embedding representations. It first converts pseudo-notes  $W_i$  into sparse text features by using the TF-IDF method, then it applies truncated SVD to transform these into dense and lower-dimensional embeddings.

**Step 1: TF-IDF Feature Construction.** Obtaining our text-formatted pseudo-notes  $W_i$ , we construct sparse TF-IDF vectors:

$$V_i = \psi(W_i) \in \mathbb{R}^m$$

where  $\psi(\cdot)$  = TF-IDF vectorizer,  $m$  = vocabulary size

**Step 2: Truncated SVD** Here, we apply truncated SVD to reduce the dimensionality of those sparse vectors, which generates a lower-dimensional embedding:

$$R_i = \text{SVD}_d(V_i)$$

Used final dimensions

$$d \in \{100, 300, 500\}$$

## 4.2 Pipeline B

Pipeline B transforms pseudo-notes  $W_i$  into semantic vector embeddings by using pretrained sentence-transformer models. Compared to Pipeline A, which mainly captures keyword frequency statistics, Pipeline B focuses more on semantic meaning and contextual relationships between household characteristics. After generating dense transformer embeddings, PCA is applied to reduce dimensionality and construct compact low-dimensional representations.

**Step 1: Sentence-Transformer Embeddings.** Given pseudo-text notes  $W_i$ , we apply a pretrained sentence-transformer encoder:

$$E_i = f_\theta(W_i) \in \mathbb{R}^{d_0}$$

where:

- $f_\theta(\cdot)$  = pretrained sentence-transformer encoder
- $d_0$  = original transformer embedding dimension

In this project:

$$d_0 \approx 384\text{--}768$$

The embeddings are then normalized using L2-normalization:

$$\tilde{E}_i = \frac{E_i}{\|E_i\|_2}$$

**Step 2: PCA Dimension Reduction.** After obtaining normalized semantic embeddings, PCA is used to reduce dimensionality and construct compact representations:

$$R_i = \text{PCA}_k(\tilde{E}_i)$$

where:

$$R_i \in \mathbb{R}^k$$

Used final dimensions:

$$k \in \{50, 100, 200\}$$

### 4.3 Pipeline C

Pipeline C focuses on Trajectory Embeddings by incorporating multiple years of historical pseudo-notes instead of using only a single-year snapshot (pipeline B). The main goal is to capture temporal household dynamics such as employment trajectories, income evolution, and long-term socio-economic patterns.

**Step 1: Construct Trajectory Text.** Instead of using only one pseudo-note  $W_i$ , we concatenate prior historical notes from multiple previous waves:

$$W_i^{\text{traj}} = \text{concat}(W_{i,t-K}, \dots, W_{i,t-1})$$

where:

- $K$  = number of historical waves (In default,  $K = 3$ )
- $W_{i,t-k}$  = pseudo-note at historical time  $t - k$

In this project, we use these 3 years' data:  $\{2007, 2009, 2011\}$

This produces a trajectory-style pseudo-note containing multi-year household history.

**Step 2: Sentence-Transformer Encoding.** The concatenated trajectory note is encoded using a pretrained sentence-transformer:

$$E_i = f_\theta(W_i^{\text{traj}})$$

where:

- $f_\theta(\cdot)$  = pretrained sentence-transformer encoder
- $E_i \in \mathbb{R}^{d_0}$  = dense trajectory embedding

The embeddings are then normalized using L2-normalization:

$$\tilde{E}_i = \frac{E_i}{\|E_i\|_2}$$

**Step 3: PCA Reduction.** After obtaining normalized trajectory embeddings, PCA is applied to reduce dimensionality:

$$R_i = \text{PCA}_k(\tilde{E}_i)$$

The final dimension values are:

$$k \in \{50, 100, 200\}$$

## 4.4 Pipeline D

Pipeline D constructs representations by combining information from multiple semantic views derived from the pseudo-notes. Instead of embedding the entire pseudo-note as a single text, this pipeline separates the note into different semantic components and then applies multiple fusion strategies. The motivation is that different parts of the pseudo-note may capture different types of socioeconomic information. For example, employment history may reflect labor market behavior, while demographic and household characteristics may capture broader household context.

#### 4.4.1 Multi-View Construction and Text Embedding

**Step 1: Construct Multi-View Pseudo-Notes.** For each observation  $i$ , the pseudo-note  $W_i$  is divided into two components.

The first component corresponds to the *work-history view*, which contains information related to employment and income history. This view includes sections such as `WorkHist` and `IncomeHist`.

The second component corresponds to the *household-context view*, which summarizes broader household characteristics. This view includes sections such as `Demographics`, `Household`, `Housing`, `Assets`, `Programs`, `Health`, and `History2007_2009`.

Formally, the two views are denoted as

$$W_i^{(1)} \quad \text{and} \quad W_i^{(2)},$$

where:

- $W_i^{(1)}$  = work-history pseudo-note
- $W_i^{(2)}$  = household-context pseudo-note

This multi-view structure allows the pipeline to separately model labor-market information and broader socioeconomic context.

**Step 2: Transformer-Based Text Embedding.** After constructing the two textual views, each view is converted into a numerical representation using a transformer-based embedding model.

Specifically, we use the pretrained sentence-transformer model `all-MiniLM-L6-v2`, which maps each input text into a dense vector of dimension 384.

Let  $f_\theta(\cdot)$  denote the embedding function parameterized by the transformer model. For each observation  $i$ , the embeddings for the two views are computed as

$$E_i^{(1)} = f_\theta(W_i^{(1)}), \quad E_i^{(2)} = f_\theta(W_i^{(2)}).$$

where:

- $E_i^{(1)}$  = embedding of the work-history view
- $E_i^{(2)}$  = embedding of the household-context view
- $E_i^{(1)}, E_i^{(2)} \in \mathbb{R}^{384}$

To reduce computational cost, the embedding matrices are cached after computation so that they can be reused across different fusion strategies without recomputing the transformer embeddings.

#### 4.4.2 Pipeline D1: Concatenation + PCA

**Step 1: Normalize and Concatenate the Two Views.** Before concatenation, each embedding vector is normalized using L2-normalization:

$$\tilde{E}_i^{(1)} = \frac{E_i^{(1)}}{\|E_i^{(1)}\|_2}, \quad \tilde{E}_i^{(2)} = \frac{E_i^{(2)}}{\|E_i^{(2)}\|_2}.$$

This normalization step ensures that the two views have comparable scale and prevents one embedding from dominating the combined representation.

The normalized embeddings are then concatenated into a single vector:

$$Z_i = [\tilde{E}_i^{(1)}, \tilde{E}_i^{(2)}].$$

Since each embedding has dimension 384, the concatenated representation has dimension 768.

**Step 2: PCA Dimension Reduction.** To reduce dimensionality and remove redundant information, principal component analysis (PCA) is applied to the concatenated embeddings.

The final representation is defined as

$$R_i = \text{PCA}_k(Z_i).$$

where:

- $R_i$  = final fused representation
- $k$  = target PCA dimension

To avoid information leakage, PCA is fitted only on the training folds and then applied to the held-out fold during cross-fitting.

**Step 3: Standardization.** Finally, the PCA outputs are standardized using z-score normalization so that each dimension has mean zero and unit variance. This improves numerical stability for downstream causal estimation.

#### 4.4.3 Pipeline D2: Learned Linear Fusion

**Step 1: Construct a Learned Fusion Representation.** Pipeline D2 replaces the fixed concatenation strategy used in D1 with a learned linear fusion of the two embeddings.

Instead of directly stacking the work-history and household-context embeddings, this approach learns two projection matrices that combine the two views into a shared low-dimensional representation.

Let:

- $A_1$  = projection matrix for the work-history embedding
- $A_2$  = projection matrix for the household-context embedding

The fused representation for observation  $i$  is defined as

$$Z_i = A_1 E_i^{(1)} + A_2 E_i^{(2)}.$$



**Step 2: Learn the Projection Matrices.** In our implementation, the projection matrices are estimated on the training fold using ridge-regularized linear regression.

The input features are the centered concatenated embeddings:

$$X_{\text{train}} = [X_{1,\text{train}}^c, X_{2,\text{train}}^c],$$

where:

- $X_{1,\text{train}}^c$  = centered work-history embeddings
- $X_{2,\text{train}}^c$  = centered household-context embeddings

The regression target  $Y$  is defined as the PCA scores obtained from the same concatenated embeddings on the training fold.

Therefore, the ridge regression model learns a stable linear mapping from the original high-dimensional embeddings to a lower-dimensional proxy representation.

**Step 3: Compute the Final Representation.** After fitting the ridge regression model, the estimated coefficient matrix is divided into two parts corresponding to the two views.

The coefficients associated with the work-history embedding form  $A_1$ , while the coefficients associated with the household-context embedding form  $A_2$ .

These matrices are then used to compute the fused representation  $Z_i$  for both training and held-out observations.

Finally, z-score normalization is applied using the training-fold mean and standard deviation so that the resulting representation has comparable scale across dimensions.

This produces a learned multi-view representation that is more flexible than simple concatenation while remaining computationally stable.

#### 4.4.4 Pipeline D3: Attention-based Fusion

**Step 1: Construct Attention Weights.** Pipeline D3 introduces a lightweight attention mechanism to dynamically determine the relative importance of the two views for each observation.

In this setting, the two inputs correspond to the embeddings from the work-history view and the household-context view. The term “lightweight” indicates that the attention weights are computed using a simple linear scoring function rather than a large neural network.

Let  $g(\cdot)$  denote a linear scoring function that maps an embedding vector to a scalar score.

For each observation  $i$ , the attention weights are computed using a softmax function:

$$\alpha_i^{(v)} = \frac{\exp\left(g(E_i^{(v)})\right)}{\exp\left(g(E_i^{(1)})\right) + \exp\left(g(E_i^{(2)})\right)}.$$

where:

- $\alpha_i^{(1)}$  = attention weight for the work-history view
- $\alpha_i^{(2)}$  = attention weight for the household-context view

These weights determine how much each view contributes to the final representation.

**Step 2: Compute the Weighted Fusion Representation.** Using the attention weights, the final representation is computed as a weighted combination of the two embeddings:

$$R_i = \alpha_i^{(1)} E_i^{(1)} + \alpha_i^{(2)} E_i^{(2)}.$$

This allows the model to dynamically emphasize the more informative view for each observation.

**Step 3: Train the Scoring Function.** In our implementation, the scoring function  $g(\cdot)$  is implemented using a ridge-regularized linear classifier.

The classifier takes an embedding vector as input and produces a scalar decision score. The training data are constructed from the embeddings of the two views:

- embeddings from the work-history view are assigned label 0
- embeddings from the household-context view are assigned label 1

The classifier is trained on the training fold using ridge regularization to improve numerical stability.

To provide weak supervision for training the scorer, a proxy label is constructed based on the relative magnitude of the raw embedding norms. For each observation, the embedding with the larger norm is treated as the more informative view and receives slightly higher training weight.

This heuristic encourages the scoring function to assign larger attention weights to views containing stronger signals.

**Step 4: Final Standardization.** After training the scoring function, the resulting scores are converted into attention weights using the softmax transformation.

These weights are then used to compute the weighted fusion representation for each observation.

Finally, the representation is standardized using z-score normalization based on the training-fold statistics to ensure comparable scale across dimensions and improve numerical stability in downstream analysis.

## 4.5 Pipeline E

A central concern when using high-dimensional text embeddings as adjustment variables in causal inference is that richer representations may inadvertently *separate* treated and control units, driving the propensity score  $\hat{e}(z) = P(T = 1 \mid Z = z)$  toward 0 or 1 and violating the positivity assumption  $0 < P(T = 1 \mid Z = z) < 1$ . This destabilizes weighted estimators (like IPW and AIPW).

Pipeline E addresses this by replacing the raw embedding  $E_i$  with a learned linear projection  $P \in \mathbb{R}^{\tau \times d}$ ,

$$Z_i = PE_i, \quad R_i = \text{zscore}(Z_i),$$

where  $P$  is trained adversarially to remove treatment-predictive directions while preserving outcome-predictive signal, maintaining overlap by construction at the cost of discarding directions that would otherwise drive positivity violations.

### Step 1: Choose a Base Embedding $E_i$

Pipeline E takes raw pseudo-note text  $W_i$  as input and encodes it directly using a sentence-transformer model (all-MiniLM-L6-v2). The resulting embeddings  $E_i \in \mathbb{R}^d$  are L2-normalized before being passed to the projection network. Embeddings are cached on disk so that the sentence-transformer is not re-run when sweeping hyperparameters.

### Step 2: Define the Adversarial Overlap Objective

The core idea is to make treatment  $T$  hard to predict from the projected representation  $Z = PE$ . We introduce a logistic classifier  $h_\omega : \mathbb{R}^k \rightarrow (0, 1)$  with parameters  $\omega$  that tries to predict  $T$  from  $Z$ :

$$h_\omega(Z) = \sigma(\omega^\top Z), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The classifier’s cross-entropy loss on the training data is

$$\mathcal{L}_{\text{ovlp}}(P, \omega) = - \sum_i i = 1^n \left[ T_i \log h_\omega(PE_i) + (1 - T_i) \log (1 - h_\omega(PE_i)) \right].$$

The adversarial training game is then

$$\min_{\omega} \mathcal{L}_{\text{ovlp}}(P, \omega) \quad \text{and} \quad \max_P \mathcal{L}_{\text{ovlp}}(P, \omega).$$

Intuitively,  $\omega$  does its best to predict  $T$  from  $Z$ , while  $P$  moves in the opposite direction, erasing the dimensions of  $E$  that the classifier finds most informative about treatment. At equilibrium,  $T$  is no longer linearly recoverable from  $Z$ , which is a necessary condition for overlap.

**Implementation via gradient reversal.** The min-max problem is typically implemented with a single backward pass using a *gradient reversal layer*: during the forward pass it is the identity, during the backward pass it multiplies the gradient by  $-\lambda_{\text{rev}}$  before passing it to  $P$ . This lets standard optimizers (Adam, SGD) jointly update  $P$  and  $\omega$  in one training loop, with  $P$  effectively maximizing the loss the classifier minimizes.

### Step 3: Add a Signal-Preservation Objective

To prevent the projection from destroying outcome-relevant signal, an outcome regressor  $m_\beta(Z)$  predicts  $Y$  from the projected representation  $Z = PE$  alone (structured covariates  $X$  are not used in the network):

$$\mathcal{L}_Y(P, \beta) = \sum_{i=1}^n (Y_i - m_\beta(PE_i))^2.$$

Before computing this loss,  $Y$  is z-scored using its training-sample mean and standard deviation so that the MSE scale does not overwhelm the adversarial BCE term.

The full training objective, minimized jointly in a single loop via gradient reversal, is:

$$\min_{P, \omega, \beta} \left[ \mathcal{L}_Y(P, \beta); +; \mathcal{L}_{\text{ovlp}}(P, \omega); +; \lambda, |P|_F^2 \right]$$

where the gradient reversal layer ensures that the gradient of  $\mathcal{L}_{\text{ovlp}}$  is negated when it flows back into  $P$ , implementing the  $\max_P$  direction without a separate update step.

The hyperparameters control three tensions:

- $\lambda \geq 0$ : ridge regularization on  $P$ , prevents over-fitting and stabilizes the projection.

- $k \in \{20, 50, 100\}$ : output dimension of the projection. Smaller  $k$  aggressively compresses, larger  $k$  retains more structure.

#### Step 4: Cross-Fitted Training Loop

To prevent leakage between representation learning and downstream causal estimation, all parameters of Pipeline E  $(P, \omega, \beta)$  must be fit on training folds only. For each fold  $k \in \{1, \dots, K\}$ :

1. Let  $\mathcal{T}_k = \bigcup_{j \neq k} I_j$  be the training indices and  $\mathcal{S}_k = I_k$  the held-out indices.
2. Load (or compute) base embeddings  $\{E_i\}_{i \in \mathcal{T}_k \cup \mathcal{S}_k}$ .
3. Initialize  $P, \omega, \beta$  (e.g.,  $P$  from PCA loadings,  $\omega, \beta$  random).
4. Run mini-batch SGD/Adam on  $\mathcal{T}_k$  alternating (or jointly via gradient reversal) the updates:

$$\begin{aligned}\omega &\leftarrow \omega - \eta \nabla_{\omega} \mathcal{L}_{\text{ovlp}}(P, \omega), \\ \beta &\leftarrow \beta - \eta \nabla_{\beta} \mathcal{L}_Y(P, \beta), \\ P &\leftarrow P - \eta \nabla_P \left[ \mathcal{L}_Y(P, \beta) + \lambda \|P\|_F^2 - \gamma \mathcal{L}_{\text{ovlp}}(P, \omega) \right].\end{aligned}$$

5. Compute  $Z_i = PE_i$  for  $i \in \mathcal{S}_k$  using the *fitted-on-train* projection  $P$ .
6. Standardize  $R_i = \text{zscore}(Z_i)$  using the training-fold mean and standard deviation only.

After looping over all folds,  $\{R_i\}_{i=1}^n$  forms the final representation, with every  $R_i$  produced by a  $P$  that never saw  $i$  during training.

#### Step 5: Hyperparameter Sweep

The sweep is small enough to run on a laptop:

- Output dimension  $k \in \{20, 50\}$  (extend to 100 if compute allows).
- Overlap weight  $\gamma \in \{0.1, 1.0, 10.0\}$  (log-spaced).
- Ridge  $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$ .
- Learning rate  $\eta \in \{10^{-3}, 10^{-4}\}$ , training steps  $\sim 1000\text{--}5000$ .

### Step 6: Overlap and Stability Diagnostics

Pipeline E must be evaluated using the same diagnostics applied to every pipeline, but with particular attention to whether the projection actually delivers its promise of improved overlap:

- Propensity-score overlap: fit a held-out propensity model  $\hat{e}(Z)$  and report the histogram of  $\hat{e}_i$  stratified by  $T_i$ , the fraction of units with  $\hat{e}_i < 0.05$  or  $\hat{e}_i > 0.95$ , and the ESS of IPW weights. A successful run should show lower extreme-propensity rates and higher ESS than the base embedding.
- Representational separability: cross-validated AUC for predicting  $T$  from  $Z$ . As  $\gamma$  increases, this should move toward 0.5.
- Outcome-signal retention: cross-validated  $R^2$  of  $\hat{Y}(Z)$  on held-out data.
- Perturbation sensitivity:  $\Delta_i = |P\tilde{E}_i - PE_i|_2$  for perturbed pseudo-notes  $\tilde{W}_i$ ; report mean and median  $\Delta_i$ .

## 4.6 Pipeline F

Pipeline F was designed to build a text representation that is stable under small, meaning-preserving changes in the pseudo-notes. In our data, the pseudo-note column  $W$  had already been constructed from pre-treatment information, so the input text was treated as fixed. The main goal of this pipeline was to reduce unnecessary sensitivity in the embedding space while keeping meaningful household-level variation.

**Step 1: Prepare the input text.** We began with the household identifier PID and the pseudo-note column  $W$  from the 2011 dataset. Each row corresponded to one household observation, and the row order was kept unchanged throughout the pipeline. This ensured that all final outputs could still be matched back to the original data after representation construction.

**Step 2: Create fold assignments.** To make the procedure fold-aware and avoid information leakage, we split the full sample into five folds using `KFold` with shuffling and a fixed random seed. These fold assignments were saved and reused in all later steps. The purpose of this step was to make sure that every learned transformation in the pipeline was fit only on training folds and then applied to the held-out fold.

**Step 3: Construct two augmented views of each pseudo-note.** For each observation  $i$ , we generated two modified versions of the same pseudo-note,

$$W_i^{(1)} = a_1(W_i), \quad W_i^{(2)} = a_2(W_i).$$

These two views were created using simple meaning-preserving transformations. Specifically, we used section-order shuffling, synonym replacement for non-technical words, numeric rounding, and punctuation or whitespace normalization. The purpose of this step was to create two texts that looked slightly different on the surface but still described the same household information.

**Step 4: Encode the augmented notes.** After creating the two text views, we encoded them using the pretrained sentence-transformer model `all-MiniLM-L6-v2`. Let

$$u_i^{(1)} = f_\theta(W_i^{(1)}), \quad u_i^{(2)} = f_\theta(W_i^{(2)})$$



denote the two resulting embeddings. The base encoder was kept frozen during training, so we did not update the transformer itself. This kept the computation manageable and made the method easier to run on a standard local machine.

**Step 5: Train a projection head with contrastive loss.** On top of the frozen encoder output, we trained a small projection head using a contrastive objective. The idea was to make the two augmented views of the same household close to each other in the learned representation space, while still keeping different households separated. After L2 normalization, the loss over a mini-batch  $\mathcal{B}$  was

$$\mathcal{L}_{\text{NCE}} = - \sum_{i \in \mathcal{B}} \log \frac{\exp(\langle \tilde{u}_i^{(1)}, \tilde{u}_i^{(2)} \rangle / \tau)}{\sum_{k \in \mathcal{B}} \exp(\langle \tilde{u}_i^{(1)}, \tilde{u}_k^{(2)} \rangle / \tau)}.$$

This step gave us a stability-tuned embedding, because the model learned to treat two harmlessly perturbed versions of the same note as similar.

**Step 6: Build the final projected embedding.** After training the projection head within each training fold, we applied the trained mapping to the original pseudo-note  $W_i$  for both training and held-out observations. In this way, each household received a projected embedding that reflected the stability-tuning step.

**Step 7: Apply fold-aware PCA.** The projected embeddings were still relatively high-dimensional, so we reduced their dimension using principal component analysis (PCA). To preserve fold awareness, PCA was fit only on the training portion of each fold and then applied to the corresponding held-out fold. After PCA transformation, the resulting features were standardized using training-fold means and standard deviations. This produced lower-dimensional representations that were easier to use in later analysis.

**Step 8: Construct the final outputs.** To study dimension sensitivity, we repeated the PCA reduction for two target dimensions,

$$k \in \{50, 100\}.$$

This produced two final representation matrices,

$$R^{(50)} \in \mathbb{R}^{n \times 50} \quad \text{and} \quad R^{(100)} \in \mathbb{R}^{n \times 100},$$

where  $n$  is the number of household observations. We saved these outputs as `pipelineF_R50.csv` and `pipelineF_R100.csv`, with PID attached for later merging and downstream analysis. We also saved the fold assignment file and a configuration file so that the pipeline could be reproduced.

**Step 9: Evaluate stability.** To check whether the learned representation actually became more stable, we measured the distance between the embeddings of the two augmented views for the same observation. For each fold, we summarized this distance using its mean and median. The mean distances were approximately 0.34 to 0.43 across folds, and the median distances were approximately 0.32 to 0.41. These results suggest that the learned embeddings were reasonably stable under small perturbations, while still preserving meaningful differences across households.

**Final result.** Overall, Pipeline F took the pre-built pseudo-note text as input, generated two augmented views for each household, trained a contrastive projection head to improve stability, and then applied fold-aware PCA to obtain reduced representations. The final result of this pipeline was two household-level text representations,  $R^{(50)}$  and  $R^{(100)}$ , together with stability diagnostics showing that the learned embeddings were consistent across small changes in text form.

## 4.7 Pipeline G

In Pipeline G, we constructed a text-based representation for each household using the pre-built pseudo-note column  $W$ . Since the dataset provided for this step already contained a leak-proof version of  $W$ , we treated the text as fixed input and focused only on representation construction.

**Step 1: Prepare the input text.** We first loaded the dataset and extracted the household identifier PID together with the pseudo-note column  $W$ . Each row in the dataset corresponded to one household observation, and the row order was kept unchanged throughout the pipeline so that the final representation matrix remained aligned with the original data.

**Step 2: Create fold assignments.** To make the representation-building process fold-aware, we partitioned the full sample into five folds using `KFold` with shuffling and a fixed random seed. These fold assignments were saved to disk so that all downstream steps could be reproduced exactly. The purpose of this step was to ensure that any learned transformation, such as PCA, was fit only on training folds and then applied to the held-out fold.

**Step 3: Encode the pseudo-notes with two sentence embedding models.** We used two pretrained sentence embedding models, `all-MiniLM-L6-v2` and `all-distilroberta-v1`, to encode the same pseudo-note  $W_i$  for each observation  $i$ . Let

$$E_i^{(1)} \in \mathbb{R}^{384} \quad \text{and} \quad E_i^{(2)} \in \mathbb{R}^{768}$$

denote the embeddings produced by the two models. Both models were applied to all rows of the dataset, and normalized embeddings were used in the encoding step.

**Step 4: Cache the transformer embeddings.** Because transformer encoding is computationally expensive, we saved the resulting embeddings to disk after they were generated.

This caching step allowed the pipeline to reload previously computed embeddings instead of recomputing them in future runs. As a result, repeated runs of the pipeline became substantially faster.

**Step 5: Combine the two embeddings by stacking.** After obtaining the two model outputs, we combined them by concatenating them along the feature dimension. For each observation  $i$ , the ensembled representation was defined as

$$E_i^{\text{stack}} = \begin{bmatrix} E_i^{(1)} & E_i^{(2)} \end{bmatrix} \in \mathbb{R}^{1152}.$$

We initially considered direct averaging, but this option was not valid in the final implementation because the two models produced embeddings with different dimensions, namely 384 and 768. Therefore, stacking was used as the final ensemble strategy.

**Step 6: Apply fold-aware PCA.** The stacked embedding was high-dimensional, so we reduced its dimension using principal component analysis (PCA). To preserve fold awareness, PCA was fit only on the training portion of each fold and then applied to the corresponding held-out fold. After PCA transformation, we standardized the transformed features using training-fold statistics before writing the result into the final output matrix. This procedure avoided fitting the reduction step on the full sample at once.

**Step 7: Construct two final representation versions.** To study dimension sensitivity, we repeated the fold-aware PCA step for two target dimensions,

$$k \in \{50, 100\}.$$

This produced two final representation matrices:

$$R^{(50)} \in \mathbb{R}^{n \times 50} \quad \text{and} \quad R^{(100)} \in \mathbb{R}^{n \times 100},$$

where  $n$  is the number of household observations. Thus, each household received two reduced versions of the stacked embedding, one with 50 dimensions and one with 100 dimensions.

**Step 8: Save all outputs.** Finally, we saved the full set of pipeline outputs to disk. These outputs included the fold assignment file, the representation matrices for  $k = 50$  and  $k = 100$ , CSV versions with PID attached for easier merging, a configuration file recording the run settings, a diagnostics file summarizing the embedding dimensions and PCA results, and the cached embedding files. Saving these artifacts made the pipeline reproducible and easier to reuse in later causal estimation steps.

**Summary.** Overall, Pipeline G used two pretrained sentence embedding models to represent the same pseudo-note text, combined the two outputs by stacking, and then applied fold-aware PCA to obtain lower-dimensional household representations. The final result was two ensembled representation versions, corresponding to  $k = 50$  and  $k = 100$ , which can be used in later overlap diagnostics and causal estimation analyses.

## 5 Semi-synthetic Benchmark Design

### 5.1 Framework Overview

To evaluate causal adjustment methods under controlled but realistic conditions, we construct a semi-synthetic benchmark based on the empirical PSID panel structure. The benchmark retains real pre-treatment household covariates from the aligned PSID sample, while treatment assignment and post-treatment outcomes are generated from a known data-generating process (DGP). This design combines the realism of observed socioeconomic data with the transparency of simulation-based ground truth.

For each eligible household-wave pair  $(i, t)$ , the benchmark contains observed pre-treatment covariates  $X_{i,t-1}$ , a binary treatment indicator  $T_{i,t}$ , potential outcomes  $Y_{i,t+h}^0$  and  $Y_{i,t+h}^1$ , and

the realized observed outcome.

$$Y_{i,t+h} = T_{i,t}Y_{i,t+h}^1 + (1 - T_{i,t})Y_{i,t+h}^0,$$

where  $h \geq 2$  denotes the post-treatment horizon. Throughout the benchmark, all adjustment variables are restricted to information measured no later than  $t - 1$ , ensuring correct causal timing.

The purpose of this framework is to test whether embedding-based representations can recover useful adjustment information when treatment selection depends on both observed and unobserved factors, and when outcomes may follow nonlinear and heterogeneous response patterns.

## 5.2 Latent Confounding and Treatment Assignment

A central feature of the benchmark is the introduction of a latent confounder  $U_{i,t-1}$ , which represents unobserved household traits such as motivation, earnings capacity, employability, or social capital. This variable affects both treatment participation and future income, but is intentionally hidden from estimation procedures.

The latent confounder is constructed from pre-treatment history features  $H_{i,t-1}$  using an index-plus-noise specification:

$$U_{i,t-1} = \text{std}(w^\top H_{i,t-1}) + \sigma_u \xi_{i,t}, \quad \xi_{i,t} \sim N(0, 1),$$

where  $w$  is a weight vector,  $\text{std}(\cdot)$  denotes standardization, and  $\sigma_u$  controls the magnitude of irreducible latent variation. Larger values of  $\sigma_u$  imply stronger hidden confounding and greater benchmark difficulty.

Treatment is then assigned probabilistically through a logistic propensity score model:

$$p_{i,t} = P(T_{i,t} = 1 \mid \tilde{X}_{i,t-1}, U_{i,t-1}) = \sigma\left(\alpha_0 + s_x \alpha_x^\top \tilde{X}_{i,t-1} + s_u \alpha_u U_{i,t-1}\right),$$

where  $\sigma(z) = \frac{1}{1+e^{-z}}$  is the logistic link function. The realized treatment is sampled as

$$T_{i,t} \sim \text{Bernoulli}(p_{i,t}).$$

In this specification,  $\alpha_0$  controls the overall treatment rate,  $s_x$  governs selection on observed covariates, and  $s_u$  controls selection on latent factors. When  $s_u = 0$ , treatment depends only on observables; larger  $s_u$  creates stronger hidden selection bias. Optional clipping of  $p_{i,t}$  to  $[\varepsilon, 1 - \varepsilon]$  is used to maintain overlap between treated and untreated groups.

### 5.3 Potential Outcome Models

Post-treatment outcomes are generated under the potential-outcomes framework. For each unit, the untreated potential outcome at horizon  $h$  is defined as

$$Y_{i,t+h}^0 = g(\tilde{X}_{i,t-1}) + \gamma_u U_{i,t-1} + \rho Y_{i,t-1}^{base} + \varepsilon_{i,t}, \quad \varepsilon_{i,t} \sim N(0, \sigma_y^2),$$

where  $g(\cdot)$  is the baseline outcome function,  $\gamma_u$  controls outcome dependence on the hidden confounder,  $\rho$  captures persistence from baseline income, and  $\varepsilon_{i,t}$  is an idiosyncratic shock.

The treated potential outcome is generated additively:

$$Y_{i,t+h}^1 = Y_{i,t+h}^0 + \tau(\tilde{X}_{i,t-1}, U_{i,t-1}),$$

where  $\tau(\cdot)$  is the treatment-effect function. This implies the unit-level causal effect is

$$\tau_{i,t} = Y_{i,t+h}^1 - Y_{i,t+h}^0 = \tau(\tilde{X}_{i,t-1}, U_{i,t-1}).$$

The observed outcome then follows:

$$Y_{i,t+h} = Y_{i,t+h}^0 + T_{i,t} \tau(\tilde{X}_{i,t-1}, U_{i,t-1}).$$

This additive structure allows transparent control of baseline income dynamics, hidden confounding, and heterogeneous treatment responses.

## 5.4 Difficulty Tiers and Treatment Heterogeneity

To study method performance under increasing complexity, the benchmark varies both the baseline function  $g(\cdot)$  and the treatment-effect function  $\tau(\cdot)$ .

**Baseline Outcome Difficulty Tiers. Tier 0 (Linear).**

$$g(\tilde{X}) = \beta^\top \tilde{X}.$$

Income depends linearly on covariates. This is the easiest setting and favors linear estimators.

**Tier 1 (Interactions).**

$$g(\tilde{X}) = \beta^\top \tilde{X} + \sum_{j < k} \beta_{jk} \tilde{X}_j \tilde{X}_k.$$

Covariates interact with one another, creating moderate model misspecification risk.

**Tier 2 (Smooth Nonlinear).**

$$g(\tilde{X}) = \beta^\top \tilde{X} + \sum_j a_j \sin(\tilde{X}_j) + \sum_j b_j \tilde{X}_j^2.$$

Income responds through curved and smooth nonlinear patterns.

**Tier 3 (Threshold / Piecewise).**

$$g(\tilde{X}) = \beta^\top \tilde{X} + c \mathbf{1}\{\tilde{X}_m < 0\} + d \max(0, \tilde{X}_n - 1).$$

Income changes abruptly at thresholds or follows piecewise responses.



### **Treatment-effect Specifications. Constant Effect.**

$$\tau(\tilde{X}, U) = \tau_0.$$

All households receive the same training benefit.

### **Linear Heterogeneity.**

$$\tau(\tilde{X}, U) = \tau_0 + \tau_x^\top \tilde{X} + \tau_u U.$$

Treatment effects vary systematically with observed and latent characteristics.

### **Nonlinear Heterogeneity.**

$$\tau(\tilde{X}, U) = \tau_0 + \tau_1 \tanh(\tilde{X}_m) + \tau_2 1\{\tilde{X}_n > 0\} + \tau_3 U^2.$$

Treatment gains vary in nonlinear and threshold-dependent ways, creating the most difficult setting.

Together, these tiers create benchmark scenarios ranging from easy linear settings to highly nonlinear environments with heterogeneous treatment responses.

## **5.5 Ground-truth Effects and Evaluation**

Because the benchmark specifies both potential outcomes, the true average treatment effect at horizon  $h$  is directly known:

$$ATE_h = \mathbb{E}[Y_{i,t+h}^1 - Y_{i,t+h}^0] = \mathbb{E}[\tau(\tilde{X}_{i,t-1}, U_{i,t-1})].$$

For the constructed finite sample, the empirical benchmark truth is

$$\widehat{ATE}_h^{truth} = \frac{1}{N} \sum_{(i,t)} \tau(\tilde{X}_{i,t-1}, U_{i,t-1}).$$

This known truth allows direct evaluation of competing estimators using metrics such as bias, RMSE, confidence interval coverage, overlap diagnostics, and robustness across benchmark difficulty settings. As a result, performance can be judged by causal accuracy rather than predictive fit alone.

## 6 Causal Estimation Methods

### 6.1 Baseline Estimators

#### 6.1.1 Difference in Means

The difference-in-means estimator is the simplest benchmark estimator used in this study. It compares the average observed outcome among treated households with the average observed outcome among untreated households, without adjusting for any covariates or embedding representations. Let  $T_i \in \{0, 1\}$  denote the treatment indicator and let  $Y_i$  denote the observed post-treatment outcome. Define

$$n_1 = \sum_{i=1}^n T_i, \quad n_0 = \sum_{i=1}^n (1 - T_i),$$

where  $n_1$  is the number of treated households and  $n_0$  is the number of untreated households.

The difference-in-means estimator is

$$\hat{\tau}_{DM} = \frac{1}{n_1} \sum_{i:T_i=1} Y_i - \frac{1}{n_0} \sum_{i:T_i=0} Y_i.$$

In words, this estimator calculates the raw gap in average outcomes between the training group and the non-training group. In our setting, it compares the average post-treatment household income for households that participated in job-related training with the average post-treatment household income for households that did not participate.

Although this estimator is easy to interpret, it does not adjust for confounding. It

treats the treated and untreated groups as if they were comparable before treatment. This assumption is generally unrealistic in observational PSID data, because households that participate in job training may differ from untreated households in prior income, education, employment history, family structure, health, or latent factors such as motivation and labor market attachment.

Therefore, the difference-in-means estimate is not used as the main causal estimator. Instead, it serves as an unadjusted baseline. Its main purpose is to show the raw association between treatment and outcome before applying any adjustment method. If the difference-in-means estimate differs substantially from adjusted estimators such as regression adjustment, IPW, or AIPW, this suggests that selection into treatment is important and that confounding adjustment is necessary.

In the semi-synthetic benchmark, this estimator is especially useful because the true treatment effect is known. We can compare  $\hat{\tau}_{DM}$  with the ground-truth ATE:

$$\text{Bias}(\hat{\tau}_{DM}) = \hat{\tau}_{DM} - \text{ATE}^{\text{truth}}.$$

A large bias for the difference-in-means estimator indicates that the simulated treatment assignment mechanism created meaningful imbalance between treated and untreated households. This makes it a useful diagnostic for confirming that the benchmark contains a nontrivial causal adjustment problem.

### 6.1.2 Regression Adjustment

Regression adjustment estimates the treatment effect by modeling the relationship between the outcome, treatment status, and pre-treatment adjustment variables. Unlike the raw difference-in-means estimator, this method attempts to control for systematic differences between treated and untreated households by conditioning on observed information.

Let  $Z_i$  denote the adjustment variable used in estimation, where  $Z_i$  may represent

structured covariates  $X_i$ , embedding-based representations  $R_i$ , or a fused feature set  $[X_i; R_i]$ . The general goal is to estimate the conditional mean outcome:

$$\mu_t(z) = \mathbb{E}[Y_i \mid T_i = t, Z_i = z], \quad t \in \{0, 1\}.$$

That is,  $\mu_1(z)$  is the expected outcome for treated households with characteristics  $z$ , and  $\mu_0(z)$  is the expected outcome for untreated households with the same characteristics.

Once these two conditional outcome models are estimated, the regression-adjusted estimator is constructed as

$$\hat{\tau}_{RA} = \frac{1}{n} \sum_{i=1}^n (\hat{\mu}_1(Z_i) - \hat{\mu}_0(Z_i)).$$

This estimator compares predicted treated and untreated outcomes for the same households and then averages those differences across the sample. Intuitively, it asks: if each household were treated versus untreated, what would the expected outcome difference be after controlling for observed characteristics?

In this project, regression adjustment is implemented using treatment-specific outcome models. Separate models are fit within the treated and untreated groups:

$$\hat{\mu}_1(z) \approx \mathbb{E}[Y \mid T = 1, Z = z], \quad \hat{\mu}_0(z) \approx \mathbb{E}[Y \mid T = 0, Z = z].$$

This two-model strategy is more flexible than imposing a single linear model with one common slope structure, because it allows the relationship between covariates and income to differ across treatment groups.

Several model classes can be used for the outcome regressions. In our implementation, we consider both regularized linear methods and flexible machine learning estimators, including Ridge regression, Lasso regression, Random Forests, and shallow neural networks. This allows the regression-adjustment estimator to capture both linear and nonlinear relationships between adjustment variables and household income.

The validity of regression adjustment depends on two key conditions. First, the adjustment variable  $Z$  must contain sufficient confounding information so that treatment assignment is conditionally ignorable given  $Z$ . Second, the conditional mean models must be estimated with adequate accuracy. If important confounders are omitted, or if the regression model is severely misspecified, the resulting estimate may remain biased.

Within this project, regression adjustment is especially informative because it can be applied under multiple adjustment regimes:

$$Z = X, \quad Z = R, \quad Z = [X; R].$$

Comparing these settings helps determine whether embedding representations preserve information relevant to future outcomes. If regression adjustment using  $R$  performs similarly to or better than  $X$ , this suggests that embeddings may encode useful proxy information for causal adjustment.

In the semi-synthetic benchmark, where the true treatment effect is known, the performance of regression adjustment can be evaluated through:

$$\text{Bias}(\hat{\tau}_{RA}) = \hat{\tau}_{RA} - \text{ATE}^{\text{truth}},$$

and through additional criteria such as RMSE, confidence interval behavior, and robustness across benchmark difficulty tiers.

### 6.1.3 Inverse Probability Weighting (IPW)

Inverse probability weighting (IPW) estimates the treatment effect by reweighting observations according to their probability of receiving the treatment they actually received. The purpose of this method is to construct a weighted pseudo-population in which treatment assignment is independent of observed adjustment variables. In such a reweighted sample, treated and untreated households become more comparable, mimicking a randomized experiment under

the standard unconfoundedness assumption.

Let  $Z_i$  denote the adjustment variable used in the propensity score model, where  $Z_i$  may be structured covariates  $X_i$ , embedding representations  $R_i$ , or a combined feature set. The propensity score is defined as

$$e(Z_i) = P(T_i = 1 \mid Z_i).$$

In practice, the propensity score is unknown and must be estimated from the data. In this project, we estimate it using logistic regression:

$$\hat{e}(Z_i) = \sigma(\beta_0 + \beta^\top Z_i), \quad \sigma(u) = \frac{1}{1 + e^{-u}}.$$

The estimated propensity score represents the predicted probability that household  $i$  participates in training given its observed adjustment variables.

Once the propensity score is estimated, each observation receives an inverse probability weight:

$$w_i = \frac{T_i}{\hat{e}(Z_i)} + \frac{1 - T_i}{1 - \hat{e}(Z_i)}.$$

This implies:

$$w_i = \frac{1}{\hat{e}(Z_i)} \quad \text{if } T_i = 1, \quad w_i = \frac{1}{1 - \hat{e}(Z_i)} \quad \text{if } T_i = 0.$$

Households that received an unlikely treatment assignment, given their characteristics, receive larger weights because they provide more information about underrepresented regions of the covariate space.

The standard IPW estimator of the average treatment effect is

$$\hat{\tau}_{IPW} = \frac{1}{n} \sum_{i=1}^n \left( \frac{T_i Y_i}{\hat{e}(Z_i)} - \frac{(1 - T_i) Y_i}{1 - \hat{e}(Z_i)} \right).$$

This estimator compares weighted average outcomes between treated and untreated

households after balancing the distribution of adjustment variables.

In finite samples, IPW can be sensitive to extreme propensity scores. If  $\hat{e}(Z_i)$  is close to 0 or 1, the corresponding weight becomes very large and can substantially increase estimator variance. To reduce this problem, our implementation applies propensity-score clipping:

$$\hat{e}(Z_i) \leftarrow \min\{\max(\hat{e}(Z_i), \varepsilon), 1 - \varepsilon\},$$

where  $\varepsilon$  is a small threshold such as 0.01. This prevents unstable weights caused by near-zero or near-one treatment probabilities.

We also report the Hájek normalized version of IPW, which replaces raw weighted totals with normalized weighted means:

$$\hat{\tau}_{Hajek} = \frac{\sum_i \frac{T_i Y_i}{\hat{e}(Z_i)}}{\sum_i \frac{T_i}{\hat{e}(Z_i)}} - \frac{\sum_i \frac{(1-T_i) Y_i}{1-\hat{e}(Z_i)}}{\sum_i \frac{1-T_i}{1-\hat{e}(Z_i)}}.$$

The Hájek form often improves finite-sample stability because it reduces the influence of a small number of highly weighted observations.

The validity of IPW depends primarily on two conditions. First, treatment assignment must be ignorable conditional on  $Z$ . Second, overlap must hold:

$$0 < e(Z_i) < 1.$$

If some households are almost always treated or almost always untreated, then meaningful reweighting becomes difficult and IPW may perform poorly.

For this reason, overlap diagnostics are an important part of our analysis. We examine the distribution of estimated propensity scores, rates of extreme probabilities, and the effective sample size:

$$ESS = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}.$$

A low effective sample size indicates that only a small number of observations dominate

the weighted estimator.

Within this project, comparing IPW performance under  $Z = X$  and  $Z = R$  is informative. If embeddings produce better overlap and more stable weights while preserving confounding information, then IPW using  $R$  may outperform IPW using traditional covariates.

## 6.2 Doubly Robust Estimators

### 6.2.1 Augmented Inverse Probability Weighting (AIPW)

Augmented inverse probability weighting (AIPW) combines regression adjustment and inverse probability weighting into a single estimator. It uses an outcome model to predict missing potential outcomes and a propensity score model to correct residual imbalance between treated and untreated groups. Because it integrates both components, AIPW is a doubly robust estimator: it remains consistent if either the outcome model or the propensity score model is correctly specified.

Let  $Z_i$  denote the adjustment variable used in estimation, where  $Z_i$  may be structured covariates  $X_i$ , embedding-based representations  $R_i$ , or a combined feature set. Define the conditional outcome regressions

$$\mu_t(Z_i) = \mathbb{E}[Y_i \mid T_i = t, Z_i], \quad t \in \{0, 1\},$$

and the propensity score

$$e(Z_i) = P(T_i = 1 \mid Z_i).$$

In practice, these quantities are unknown and must be estimated from the data. Let

$$\hat{\mu}_1(Z_i), \quad \hat{\mu}_0(Z_i), \quad \hat{e}(Z_i)$$

denote the fitted treated outcome model, untreated outcome model, and estimated propensity score, respectively.



The AIPW estimator of the average treatment effect is

$$\hat{\tau}_{AIPW} = \frac{1}{n} \sum_{i=1}^n \left[ \hat{\mu}_1(Z_i) - \hat{\mu}_0(Z_i) + \frac{T_i\{Y_i - \hat{\mu}_1(Z_i)\}}{\hat{e}(Z_i)} - \frac{(1 - T_i)\{Y_i - \hat{\mu}_0(Z_i)\}}{1 - \hat{e}(Z_i)} \right].$$

This expression contains two parts. The first part,

$$\hat{\mu}_1(Z_i) - \hat{\mu}_0(Z_i),$$

is the regression-adjustment component. It estimates the treatment effect by comparing predicted treated and untreated outcomes for the same household.

The second part,

$$\frac{T_i\{Y_i - \hat{\mu}_1(Z_i)\}}{\hat{e}(Z_i)} - \frac{(1 - T_i)\{Y_i - \hat{\mu}_0(Z_i)\}}{1 - \hat{e}(Z_i)},$$

is the augmentation term. It reweights residual prediction errors using inverse propensity scores. If the regression model is imperfect, this correction helps remove remaining bias caused by imbalance between treated and untreated groups.

The doubly robust property can be summarized as follows:

- If the outcome regressions  $\hat{\mu}_1(Z)$  and  $\hat{\mu}_0(Z)$  are correctly specified, AIPW remains consistent even if the propensity score model is misspecified.
- If the propensity score model  $\hat{e}(Z)$  is correctly specified, AIPW remains consistent even if the outcome regressions are misspecified.
- If both nuisance models are well estimated, AIPW is typically more efficient than using only regression adjustment or only IPW.

In this project, separate outcome models are estimated for treated and untreated households, while the propensity score is estimated using logistic regression. Depending on the experiment, nuisance models may use linear methods or more flexible machine learning

approaches. Propensity scores are clipped away from 0 and 1 to avoid unstable inverse weights.

AIPW is especially valuable in the present setting because learned embedding representations may capture some aspects of confounding well but may not perfectly model treatment selection or outcomes on their own. By combining both nuisance components, AIPW provides a more reliable assessment of whether embeddings contain valid adjustment information.

Within the semi-synthetic benchmark, AIPW performance is evaluated by comparing estimated effects with the known ground-truth ATE:

$$\text{Bias}(\hat{\tau}_{AIPW}) = \hat{\tau}_{AIPW} - ATE^{truth}.$$

Additional evaluation metrics include RMSE, confidence interval coverage, and robustness across different benchmark difficulty tiers. If AIPW based on  $R$  performs comparably to or better than AIPW based on  $X$ , this suggests that embedding representations successfully preserve information relevant for causal adjustment.

### 6.2.2 Double Machine Learning (DML)

Double machine learning (DML) extends doubly robust estimation to settings where nuisance functions are estimated using flexible machine learning models. When the adjustment variable is high-dimensional or contains nonlinear structure, directly substituting fitted machine learning models into standard estimators can introduce overfitting bias. DML addresses this problem through orthogonal score construction and cross-fitting, allowing modern predictive methods to be used while maintaining reliable treatment effect estimation.

Let  $Z_i$  denote the adjustment variable used in estimation, where  $Z_i$  may represent structured covariates  $X_i$ , embedding representations  $R_i$ , or a combined feature set. Define the nuisance functions as the propensity score

$$e(Z_i) = P(T_i = 1 \mid Z_i),$$

and the conditional mean outcome models

$$\mu_1(Z_i) = \mathbb{E}[Y_i \mid T_i = 1, Z_i], \quad \mu_0(Z_i) = \mathbb{E}[Y_i \mid T_i = 0, Z_i].$$

These quantities are unknown and must be estimated from the data. Let

$$\hat{e}(Z_i), \quad \hat{\mu}_1(Z_i), \quad \hat{\mu}_0(Z_i)$$

denote the fitted nuisance components.

DML estimates the average treatment effect using an orthogonal score function. In the binary-treatment setting considered here, the estimating equation is

$$\frac{1}{n} \sum_{i=1}^n \left[ \hat{\mu}_1(Z_i) - \hat{\mu}_0(Z_i) + \frac{T_i \{Y_i - \hat{\mu}_1(Z_i)\}}{\hat{e}(Z_i)} - \frac{(1 - T_i) \{Y_i - \hat{\mu}_0(Z_i)\}}{1 - \hat{e}(Z_i)} - \tau \right] = 0.$$

Solving for  $\tau$  gives the DML estimator:

$$\hat{\tau}_{DML} = \frac{1}{n} \sum_{i=1}^n \left[ \hat{\mu}_1(Z_i) - \hat{\mu}_0(Z_i) + \frac{T_i \{Y_i - \hat{\mu}_1(Z_i)\}}{\hat{e}(Z_i)} - \frac{(1 - T_i) \{Y_i - \hat{\mu}_0(Z_i)\}}{1 - \hat{e}(Z_i)} \right].$$

Although this formula resembles AIPW, the main distinction is that DML uses cross-fitted nuisance estimates rather than in-sample fitted values.

The cross-fitting procedure divides the sample into  $K$  folds. For each fold, nuisance models are trained on the remaining  $K - 1$  folds and then used to predict propensity scores and conditional outcomes for the held-out fold. This process is repeated across all folds until every observation receives out-of-sample predictions:

$$\hat{e}_i^{cf}, \quad \hat{\mu}_{1,i}^{cf}, \quad \hat{\mu}_{0,i}^{cf}.$$

The final estimator is then computed using these cross-fitted predictions. Because each observation is evaluated using models that were not trained on that same observation, cross-fitting helps reduce overfitting bias and improves finite-sample performance.

DML is particularly useful in this project because some adjustment sets may be high-dimensional, especially when  $Z = R$  or  $Z = [X; R]$ . Embedding-based representations can contain nonlinear relationships that are difficult to capture using simple parametric models. DML allows nuisance models to be estimated using flexible methods such as regularized regression, random forests, gradient boosting, or neural networks.

Compared with standard AIPW using in-sample nuisance fits, DML often performs more reliably when flexible learners are used. It helps reduce regularization bias and makes treatment effect estimation less sensitive to overfitting in the nuisance stage.

Within the semi-synthetic benchmark, DML is evaluated by comparing its estimated treatment effect with the known ground-truth ATE:

$$\text{Bias}(\hat{\tau}_{DML}) = \hat{\tau}_{DML} - ATE^{truth}.$$

Additional evaluation metrics include RMSE, confidence interval coverage, overlap diagnostics, and repeated-seed stability. Strong DML performance using embedding representations would suggest that the learned features preserve useful confounding information for causal adjustment.

## 7 Results

### 7.1 Semi-Synthetic Monte Carlo Benchmarks

#### 7.1.1 Embeddings Consistently Outperform Covariates on Point Accuracy

Across all four tiers and all three outcome scenarios (Constant, Linear, Nonlinear), the embeddings-only condition (**R**) achieves substantially lower RMSE than the covariate-only

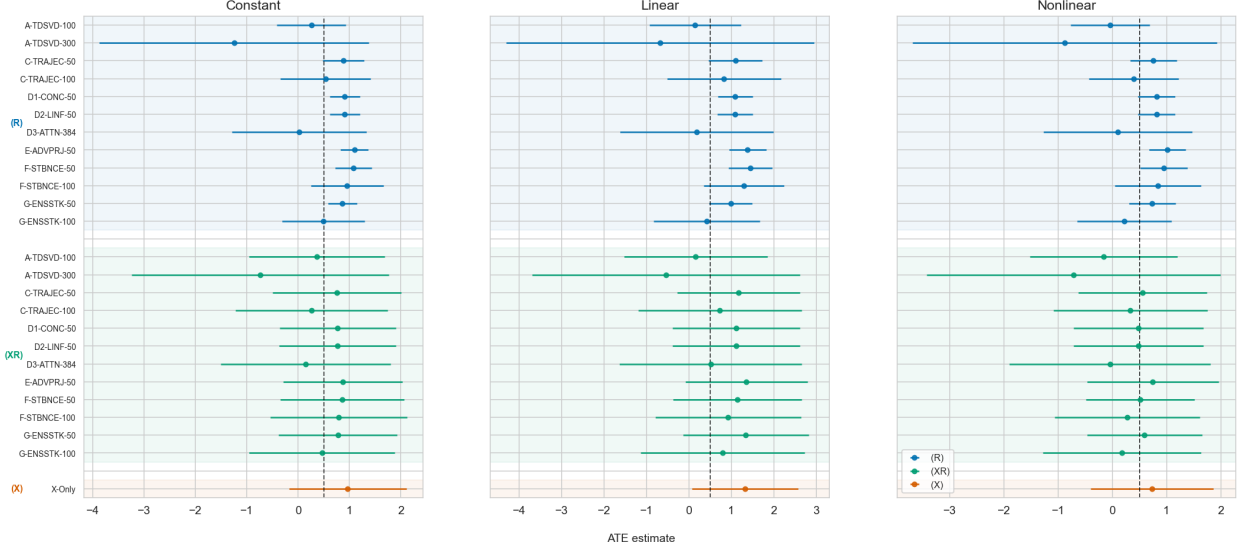
baseline **(X)** and the concatenated condition **(XR)**. This result holds without exception: no tier, scenario, or estimator combination produces a reversal where **(X)** matches the best **(R)** pipeline on point accuracy.

**Table 3:** Best **(R)**-only RMSE versus covariate-only RMSE per tier. RMSE averaged over three outcome scenarios and four AIPW estimators. Reduction =  $\text{RMSE}_X / \text{RMSE}_{R,\text{best}}$ .

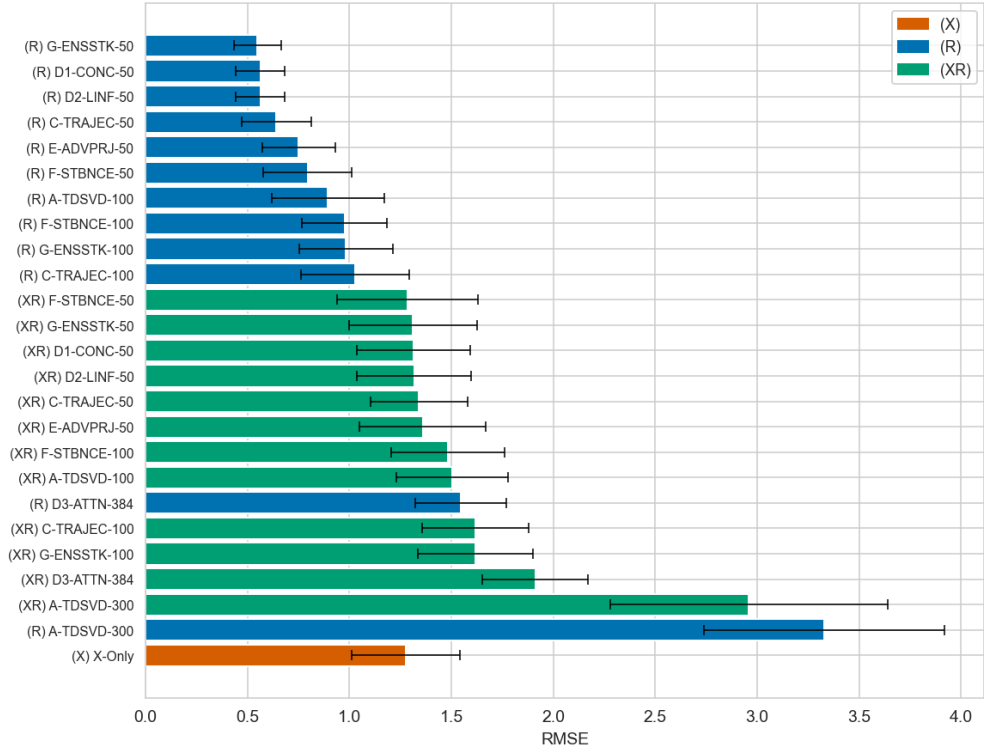
| Tier | Best <b>(R)</b> RMSE | Best pipeline | <b>(X)</b> RMSE | Reduction |
|------|----------------------|---------------|-----------------|-----------|
| 0    | 0.44                 | G-ENSSTK-50   | 1.12            | 2.5×      |
| 1    | 0.58                 | D2-LINF-50    | 1.95            | 3.4×      |
| 2    | 0.50                 | G-ENSSTK-50   | 1.21            | 2.4×      |
| 3    | 0.55                 | G-ENSSTK-50   | 1.28            | 2.3×      |

The **(R)** advantage is most pronounced in Tier 1, where the treatment–covariate interaction inflates RMSE across all conditions and disproportionately harms the **(X)** baseline (1.95 vs. 1.37 for **(R)**). The pipeline leaderboard is also highly stable across outcome scenarios within each tier, the ranking under the three outcome scenarios agrees almost perfectly with each other, meaning the embedding advantage is a property of pipeline architecture rather than scenario-specific luck.

Figure 2 shows the full RMSE leaderboard for Tier 3, and Figure 1 shows the per-condition mean ATE across scenarios.



**Figure 1:** Mean estimated ATE per pipeline, split by outcome scenario (Constant, Linear, Nonlinear), Tier 3. The dashed vertical line marks  $\tau = 0.5$ . Blue (R) markers for  $k = 50$  pipelines cluster tightly around the true ATE in all scenarios. Orange (X) markers drift in the Linear and Nonlinear scenarios, reflecting unremoved residual confounding. Green (XR) markers are centred near  $\tau$  but with wider error bars reflecting higher estimator variance.



**Figure 2:** RMSE leaderboard for Tier 3. Pipelines are sorted by mean RMSE; X-Only is the reference baseline. Low-dimensional (R) pipelines ( $k = 50$ ) occupy the top; high-dimensional embeddings and all (XR) rows occupy the bottom. The same pattern holds for Tiers 0 - 2.

### 7.1.2 Low-Dimensional PCA Embeddings ( $k = 50$ ) Dominate; High Dimensions Are Catastrophic

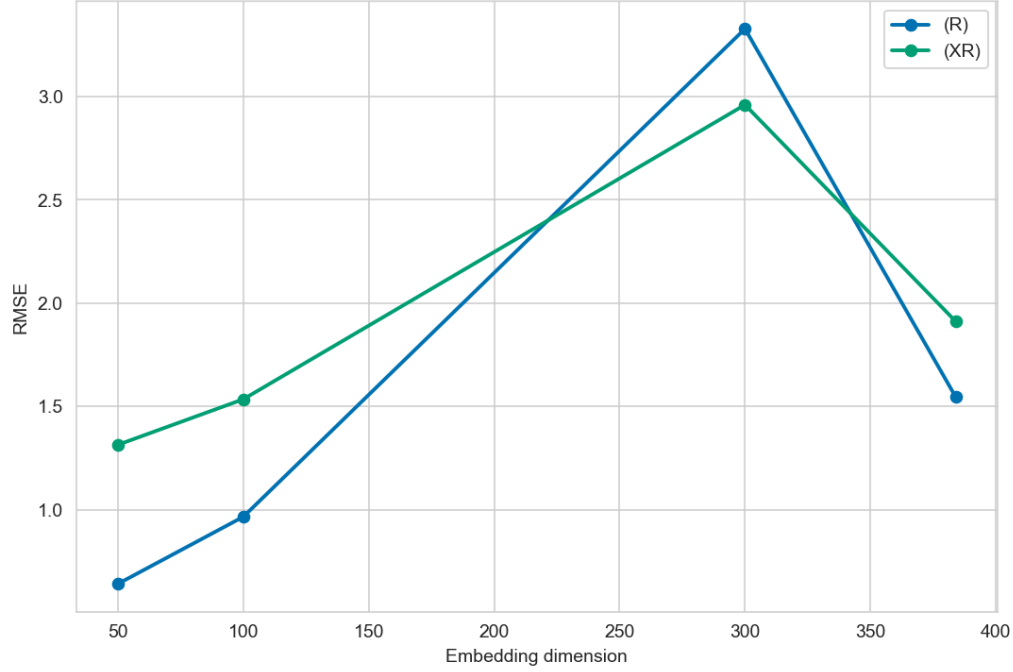
Embedding dimension is the single strongest predictor of pipeline quality, more predictive than embedding family, fusion strategy, or estimator choice. Every leaderboard across all four tiers is monotonically ordered by dimension:  $k = 50$  pipelines occupy the top,  $k = 100$  the middle, and  $k \geq 300$  the bottom.

**Table 4:** Mean (R)-only RMSE by embedding dimension across pipelines.

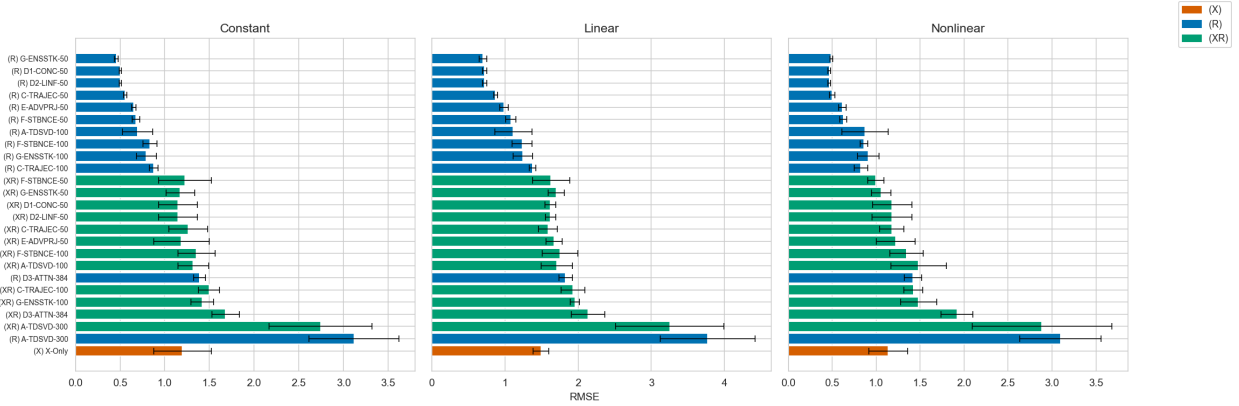
| Condition   | $k = 50$ mean RMSE | $k = 100$ mean RMSE | $k \geq 300$ mean RMSE    |
|-------------|--------------------|---------------------|---------------------------|
| Tier 0, (R) | $\approx 0.52$     | $\approx 0.80$      | $\approx 2.0\text{--}3.3$ |
| Tier 1, (R) | $\approx 0.64$     | $\approx 1.10$      | $\approx 2.2\text{--}4.1$ |
| Tier 2, (R) | $\approx 0.58$     | $\approx 0.85$      | $\approx 2.2\text{--}3.4$ |
| Tier 3, (R) | $\approx 0.60$     | $\approx 0.90$      | $\approx 2.5\text{--}3.3$ |

The two worst pipelines: A-TDSVD-300 (TF-IDF SVD,  $d = 300$ ) and D3-ATTN-384 (attention embedding,  $d = 384$ ), appear at the bottom of every leaderboard in every tier. A-TDSVD-300 achieves RMSE of 3.3–4.1 depending on tier, versus  $\approx 0.5$  for the best  $k = 50$  pipelines.

Figure 3 shows the dimension–RMSE curve for Tier 3. The relationship is monotone and steep, confirming that pipeline selection based on embedding dimension alone recovers most of the leaderboard ordering.

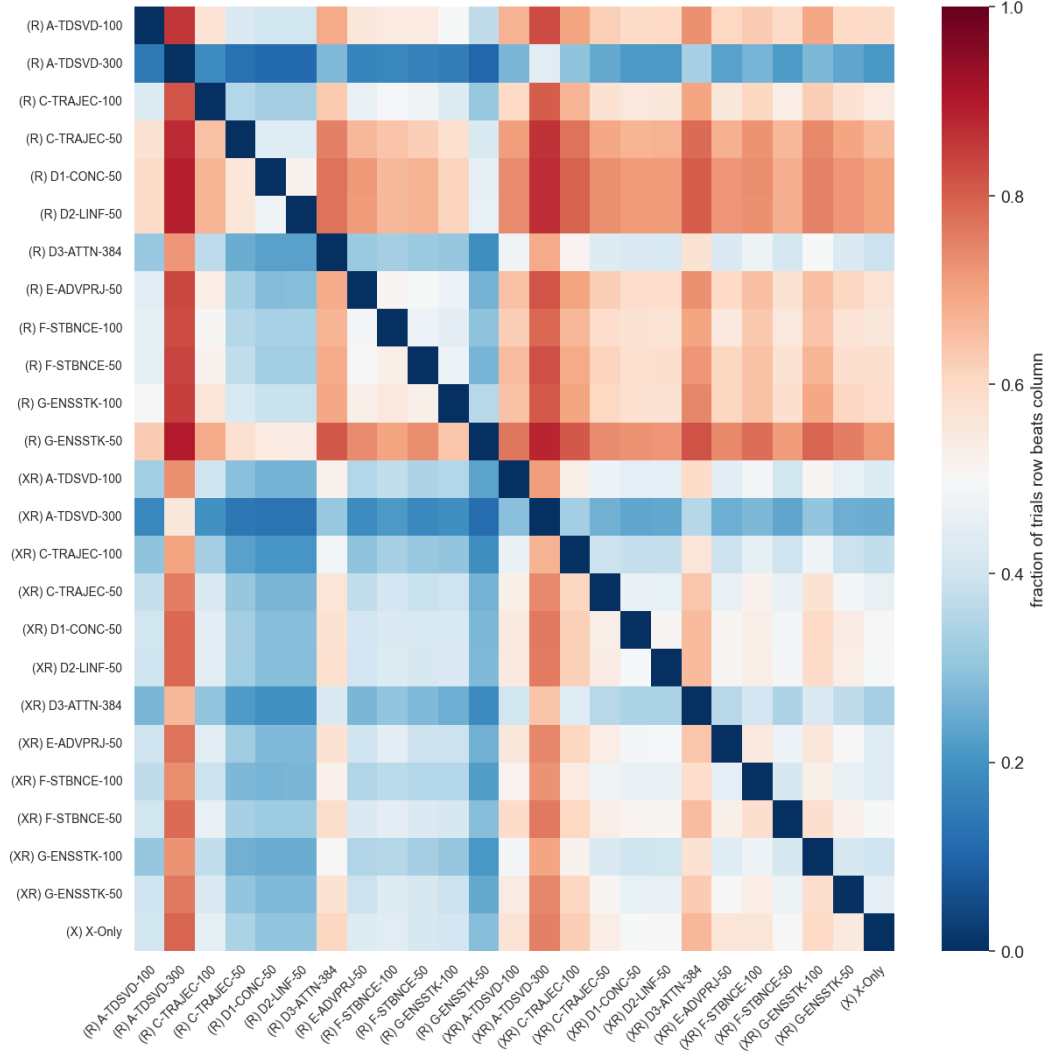


**Figure 3:** Mean RMSE as a function of embedding dimension for the (R)-only condition (Tier 3). RMSE rises monotonically with dimension:  $k = 50$  pipelines cluster in the lower-left;  $k = 384$ –300 pipelines occupy the upper-right. The same monotone relationship holds in Tiers 0, 1, and 2.



**Figure 4:** Pipeline RMSE stratified by outcome scenario (Constant, Linear, Nonlinear) for Tier 3. The leaderboard is highly stable across scenarios indicating that architecture drives performance.





**Figure 5:** Pairwise dominance matrix for Tier 3. Each cell  $(i, j)$  is the fraction of draws in which pipeline  $i$  achieves lower  $|\text{bias}|$  than pipeline  $j$ . Dark blue rows indicate dominant pipelines; dark red rows indicate near-universal losers. The (R)  $k = 50$  cluster (G-ENSSTK-50, D1-CONC-50, D2-LINF-50) forms the dominant block.

### 7.1.3 Concatenating Covariates Degrades Point Accuracy but Improves Coverage

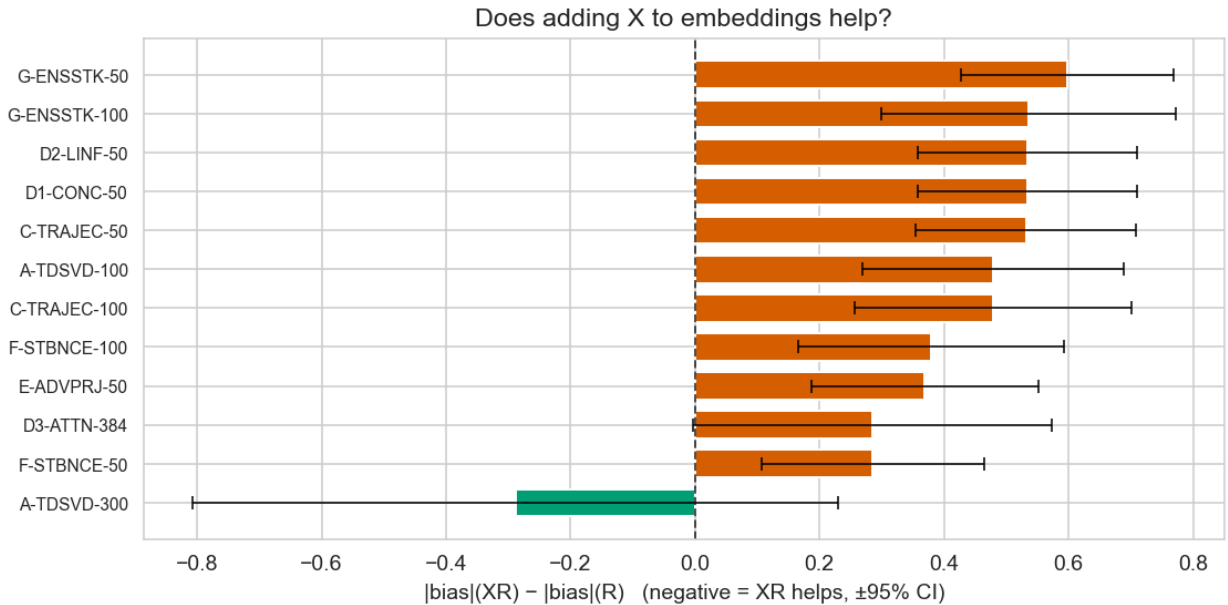
The (XR) condition consistently *worsens* point RMSE relative to (R) alone for every competitive pipeline, while simultaneously achieving the highest empirical CI coverage of all three conditions.

**Effect on point accuracy.** Table 5 reports the mean absolute-bias penalty from adding  $X$  to each pipeline’s embeddings, with the single systematic exception.

**Table 5:** Mean increase in absolute bias when moving from (R) to (XR) condition per tier. Negative values indicate (XR) helps. All values averaged over scenarios and estimators.

| Tier | $\Delta$ bias range, best pipelines | Only pipeline where XR helps     |
|------|-------------------------------------|----------------------------------|
| 0    | +0.13 to +0.33                      | A-TDSVD-300 ( $\Delta = -0.55$ ) |
| 1    | +0.28 to +0.86                      | A-TDSVD-300 ( $\Delta = -0.57$ ) |
| 2    | +0.23 to +0.74                      | A-TDSVD-300 ( $\Delta = -0.51$ ) |
| 3    | +0.28 to +0.60                      | A-TDSVD-300 ( $\Delta = -0.29$ ) |

The exception (A-TDSVD-300) is the only pipeline whose (R) embedding is so high-dimensional and noisy that adding  $X$  recovers useful identifying information. For every other pipeline, the embeddings already capture the confounding signal; concatenating  $X$  introduces collinear features that destabilise nuisance models without providing new information.

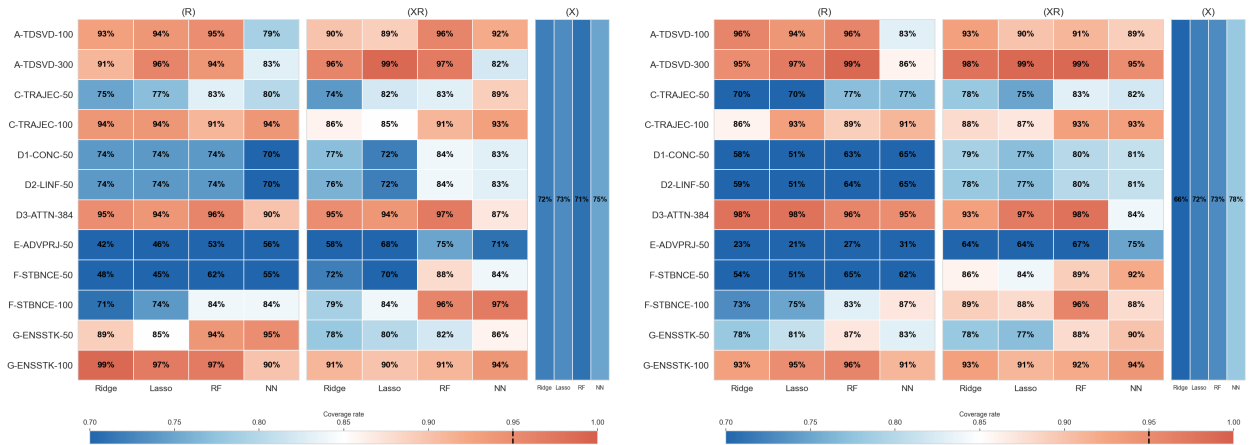


**Figure 6:** Mean of  $\text{bias}_{\text{XR}} - \text{bias}_{\text{R}}$  per pipeline for Tier 3, with 95% bootstrap CIs. Positive (red) bars indicate that adding  $X$  worsens the point estimate; negative (green) bars indicate improvement. Only A-TDSVD-300 is negative. The 95% CIs exclude zero for most large-positive-delta pipelines, confirming this is not sampling noise.

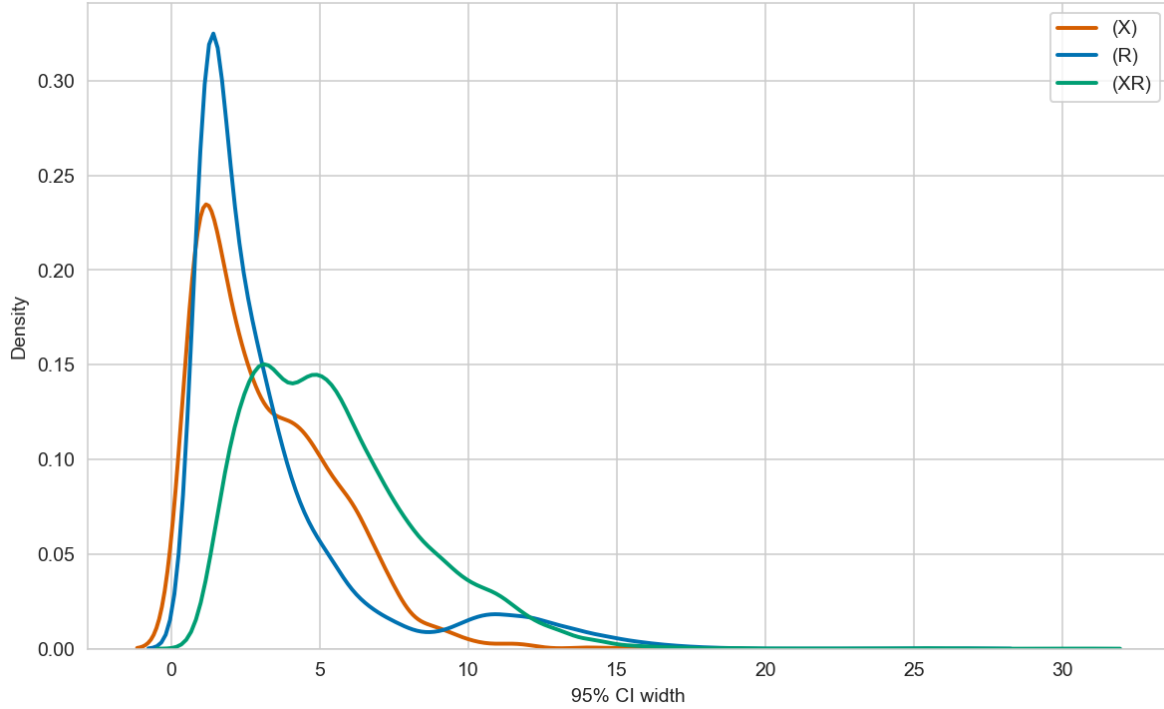
**Effect on CI coverage.** Despite worse point accuracy, (XR) consistently achieves the highest empirical 95% CI coverage across all tiers:

**Table 6:** Empirical 95% CI coverage and mean CI width by condition and tier. Coverage averaged over all pipelines, three scenarios, and four estimators.

| Tier | (R) coverage | (X) coverage | (XR) coverage | XR/R CI width ratio |
|------|--------------|--------------|---------------|---------------------|
| 0    | 0.83         | 0.66         | 0.90          | 1.34×               |
| 1    | 0.86         | 0.79         | 0.91          | 1.61×               |
| 2    | 0.81         | 0.77         | 0.87          | 1.61×               |
| 3    | 0.81         | 0.85         | 0.92          | 1.61×               |



**Figure 7:** Empirical 95% CI coverage grids for Tier 2 under the Constant (left) and linear (right) scenarios. Each cell shows coverage for one pipeline × estimator combination, colour-coded: red < 0.80, orange 0.80–0.90, yellow 0.90–0.95, green ≥ 0.95.



**Figure 8:** Kernel density of bootstrap 95% CI width by condition (Tier 2). (R) is tightly concentrated at  $\approx 3.5$  (narrow, low-variance intervals). (XR) is shifted right to  $\approx 5.75$  with a heavy right tail. (X) falls in between.

#### 7.1.4 AIPW with Random Forest Nuisance Models Is the Most Robust Estimator

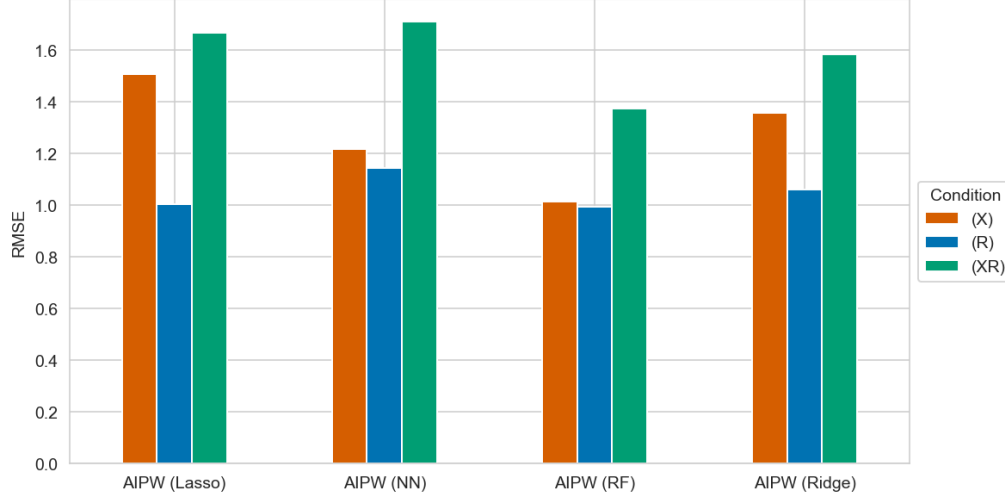
Across all four tiers, three outcome scenarios, and all covariate conditions, AIPW with Random Forest (RF) nuisance models achieves the lowest mean RMSE.

**Table 7:** Mean RMSE by AIPW estimator (averaged over all pipelines and conditions) per tier.

| Tier | RF          | Ridge | Lasso | NN   |
|------|-------------|-------|-------|------|
| 0    | <b>0.97</b> | 1.07  | 1.06  | 1.34 |
| 1    | <b>1.64</b> | 1.80  | 1.81  | 1.90 |
| 2    | <b>1.21</b> | 1.34  | 1.33  | 1.50 |
| 3    | <b>1.18</b> | 1.33  | 1.34  | 1.42 |

The RF advantage is largest in Tier 0 (28% improvement over NN) and smallest in Tier 1

(14%). RF is also the most *consistent* estimator across pipeline choices. it achieves the lowest standard deviation of absolute bias across pipelines in every tier, meaning performance is least sensitive to which embedding pipeline is supplied.



**Figure 9:** Mean RMSE per AIPW estimator (Tier 3). RF achieves the lowest RMSE in all conditions. Ridge and Lasso are nearly tied. The ordering is consistent across Tiers 0–3.

### 7.1.5 Practical Recommendations from the Study

1. **Point estimate (additive DGP):** use AIPW (RF) with G-ENSSTK-50 in the (R)-only condition. RMSE: 0.44–0.55 across Tiers 0, 2, and 3, a 2.3–2.5 $\times$  improvement over the covariate-only baseline.
2. **Confidence interval:** use AIPW (RF) with any competitive  $k = 50$  pipeline in the (XR) condition. Accept the 34–61% CI-width premium over (R)-only as the cost of higher coverage.
3. **Never use  $k \geq 300$  embeddings** (A-TDSVD-300, D3-ATTN-384) at  $n \approx 6,000$ . They are the worst performers in every tier on both RMSE and bias, with RMSE up to 10 $\times$  worse than the best  $k = 50$  option.
4. **Use AIPW (RF) as the default estimator.** Lowest RMSE in every tier and condition, most consistent across pipeline choices, least harmed by switching between

(R) and (XR).

## 7.2 Analysis of the PSID Panel Data

### 7.2.1 Dataset Description and Experimental Design

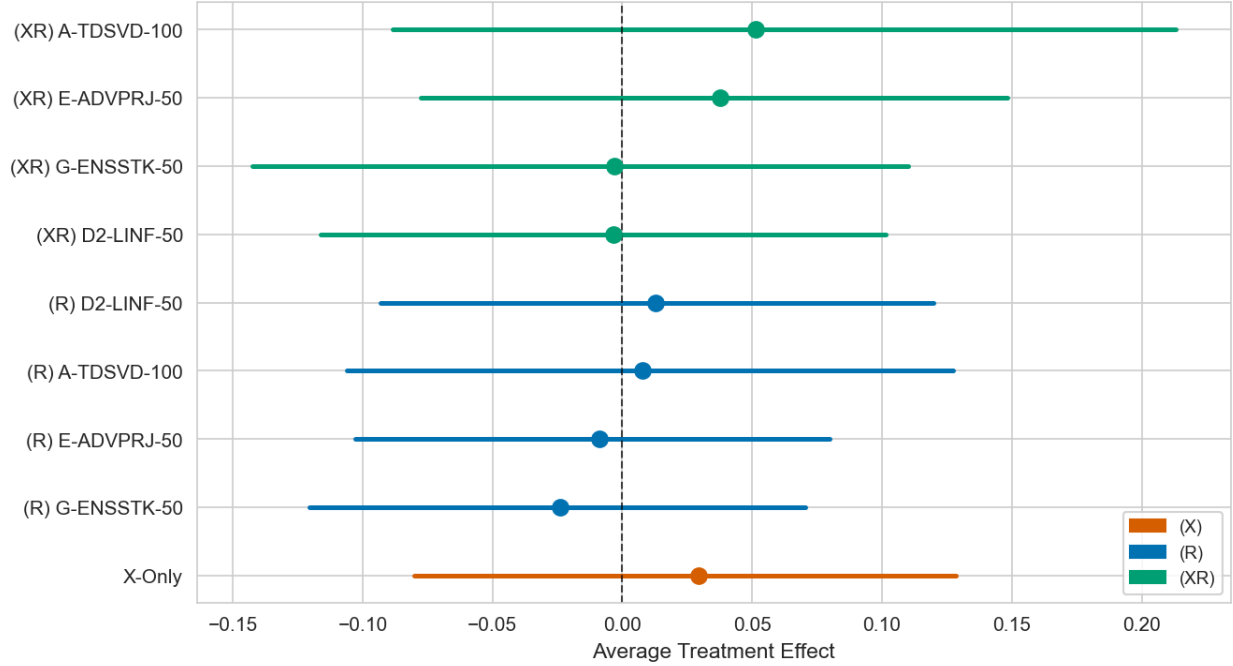
The real-data analysis uses the **Panel Study of Income Dynamics (PSID)**, a long-running longitudinal household survey covering economic, social, and demographic outcomes in the United States. After merging the structured panel covariates with the text-embedding pipelines and removing observations with missing treatment or outcome values, the analytic sample contains  $n = 6,107$  families. The treatment rate is approximately 20%.

Each pipeline is evaluated in R-only and X+R modes. All ATEs are estimated with AIPW (Ridge, Lasso, RF, NN), bootstrap CIs ( $B = 500$  resamples), across five independent random seeds  $\{42, 7, 123, 2026, 99\}$ .

### 7.2.2 Results

**Key finding.** *No condition yields a statistically significant treatment effect.* Every 95% CI contains zero, regardless of estimator, pipeline, or feature set. This finding is internally consistent across all five seeds and all pipelines.

No embedding pipeline or feature combination yields a statistically significant estimate of the treatment effect. All 95% CIs include zero. This is substantively plausible, labor-market program effects are often small and heterogeneous. G-ENSSTK-50 and E-ADVPRJ-50 are the most stable and precise ( $SD \leq 0.006$ ), consistent with the above study.



**Figure 10:** Forest plot of ATE estimates for AIPW (RF) across all pipelines. Points mark the mean ATE; bars span the mean 95% bootstrap CI. All CIs include zero — no pipeline detects a significant effect.

## 8 Conclusions and Recommendations

Our project studied whether text-based embeddings can be used as adjustment variables for causal effect estimation in the PSID setting. The main application was the effect of job-related training on later household income. Because training participation is not randomly assigned, treated and untreated households may differ before treatment in income history, education, employment status, assets, health, and other household characteristics. Some relevant factors, such as motivation, labor market attachment, and long-term earning potential, may also be only partially observed. For this reason, adjustment based only on structured covariates  $X$  may not fully capture the confounding structure.

To address this problem, we converted strictly pre-treatment household information into pseudo-text  $W$ , and then transformed the text into embedding representations  $R = \phi(W)$ . These representations were compared with structured covariates under three adjustment

regimes:  $X$ -only,  $R$ -only, and the fused representation  $[X; R]$ . The semi-synthetic benchmark was useful because the true treatment effect was known by construction, which allowed us to evaluate each method using bias, RMSE, confidence interval coverage, and overlap diagnostics.

The main result is that the embedding representation improved point estimation accuracy in the benchmark. Across all four tiers,  $R$ -only adjustment had lower RMSE than the  $X$ -only baseline. The best  $R$ -only RMSE was 0.44 in Tier 0 compared with 1.12 for  $X$ -only, 0.58 in Tier 1 compared with 1.95, 0.50 in Tier 2 compared with 1.21, and 0.55 in Tier 3 compared with 1.28. This means that the best embedding representation reduced RMSE by about 2.3 to 3.4 times relative to structured covariates alone. The largest improvement appeared in Tier 1, where the data-generating process included covariate interactions. This pattern suggests that embeddings were useful for summarizing combinations of household characteristics, rather than only using each covariate separately.

However, the results also show that lower RMSE is not the only goal in causal inference. The  $R$ -only regime gave the best point accuracy, but the fused  $[X; R]$  regime provided better confidence interval coverage in some settings. This creates a tradeoff between point estimation and inference. If the goal is to estimate the treatment effect as accurately as possible in the benchmark,  $R$ -only performs best. If the goal is more reliable uncertainty coverage, the fused representation may be more appropriate. Therefore, our results do not imply that one adjustment regime is always best. Instead, embedding-based adjustment should be judged using both accuracy measures and inference diagnostics.

For the real PSID application, the estimated average treatment effect of job-related training on later household income was statistically indistinguishable from zero across regimes and estimators. Since the real data do not reveal the true potential outcomes, this result should not be interpreted as proof that the true effect is exactly zero. A more careful interpretation is that, after adjustment, we do not find statistically significant evidence that job-related training changes later total household income in this sample.



Overall, the results suggest that pre-treatment text embeddings can be useful for causal adjustment. They reduce estimation error in the semi-synthetic benchmark and appear to capture information from household histories that is not fully represented by the original covariates. At the same time, embeddings should not be used as automatic controls. They need to be constructed from strictly pre-treatment information, checked for leakage, and evaluated with overlap and uncertainty diagnostics. Based on these findings, we recommend using embeddings as part of a full causal workflow, rather than treating them as ordinary machine learning features.

## 9 Limitations and Future Work

There are several limitations to this study. First, the RMSE improvement under the  $R$ -only regime does not directly prove that embeddings improve ignorability. One interpretation is that the embedding representation provides a better proxy for latent confounding. However, other explanations are also possible. For example, the lower-dimensional embedding space may make nuisance models easier to estimate, or the PCA step may add regularization that improves finite-sample performance. Therefore, the results provide evidence that embeddings are useful for adjustment in this benchmark, but they do not prove that embeddings fully recover the latent confounder or remove all residual confounding.

Second, the semi-synthetic benchmark is still based on a specific data-generating process. The benchmark includes several useful difficulty levels, including linear, interaction, nonlinear, and threshold settings. However, the conclusions are still tied to this design. Results may not fully generalize to more complex DGPs, such as time-varying confounding or non-additive treatment effects. Future work could extend the benchmark to these settings to test whether the advantage of embeddings remains stable.

Third, the embedding pipeline is not unique. The results may depend on the pseudo-note template, the encoder, the embedding dimension, and the dimensionality reduction

method. Although we compared several pipelines, including TF-IDF, transformer-based embeddings, trajectory embeddings, multi-view fusion, stability-tuned embeddings, and ensemble embeddings, there is still room for more sensitivity analysis. Future work could compare additional encoders, alternative pseudo-text schemas, different dimensions, and other cross-fitting strategies.

Another useful direction is uncertainty propagation. In the current workflow, the embedding representation is constructed first and then used as an input to the causal estimator. This treats the embedding as fixed, even though the representation learning step also introduces uncertainty. Future work could use Bayesian uncertainty propagation to carry this uncertainty into the final causal confidence intervals. This would give a more complete view of uncertainty in embedding-based causal inference.

In summary, Our project shows that embeddings of strictly pre-treatment household histories can improve point estimation in the semi-synthetic PSID benchmark. The main benefit appears in lower RMSE, while the main caution is the coverage tradeoff between  $R$ -only and  $[X; R]$ . Future work should focus on stronger sensitivity analysis, broader benchmark designs, and uncertainty-aware causal estimation.

## 10 Acknowledgments

We would like to thank Professor Jelena Bradic for her guidance and support throughout this project. Her feedback helped us better understand the project goal and organize the workflow from PSID data construction to pseudo-text generation, embedding pipelines, and causal estimation. Her suggestions were also helpful for keeping the analysis focused on pre-treatment information and avoiding data leakage.

We also want to thank all group members for their work and cooperation during the project. Each person contributed to different parts of the project, including data preparation, code implementation, result checking, figure generation, poster preparation, and report

writing. Working together helped us improve the final analysis and made the project more organized and complete.

Overall, this project gave us a useful opportunity to connect structured survey data, text-based representations, and causal inference methods. Through the process, we learned not only how to build a reproducible dataset and generate embeddings, but also how important careful design choices are when using machine learning tools in empirical research.

## References

- [1] Fang, X., Xu, W., Tan, F. A., Hu, Z., Zhang, J., Qi, Y., Sengamedu, S. H., & Faloutsos, C. (2024). Large Language Models (LLMs) on Tabular Data: Prediction, Generation, and Understanding – A Survey. *Transactions on Machine Learning Research*. <https://openreview.net/forum?id=IZnrCGF9WI>
- [2] Survey Research Center, Institute for Social Research, University of Michigan. *Panel Study of Income Dynamics, public use dataset*. Ann Arbor, MI: Institute for Social Research, University of Michigan. <https://psidonline.isr.umich.edu/>
- [3] Roberts, M. E., Stewart, B. M., & Nielsen, R. A. (2020). Adjusting for Confounding with Text Matching. *American Journal of Political Science*, 64(4), 887–903. <https://doi.org/10.1111/ajps.12526>
- [4] Rubin, D. B. (2005). Causal Inference Using Potential Outcomes: Design, Modeling, Decisions. *Journal of the American Statistical Association*, 100(469), 322–331. <https://doi.org/10.1198/016214504000001880>
- [5] Veitch, V., Sridhar, D., & Blei, D. (2020). Adapting Text Embeddings for Causal Inference. In J. Peters & D. Sontag (Eds.), *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence*, Proceedings of Machine Learning Research, Vol. 124, pp. 919–928. PMLR. <https://proceedings.mlr.press/v124/veitch20a.html>
- [6] Wager, S. (2024). *Causal Inference: A Statistical Learning Approach*. Stanford University. [https://web.stanford.edu/~swager/causal\\_inf\\_book.pdf](https://web.stanford.edu/~swager/causal_inf_book.pdf)

# Software Code

## Final SQL Code for PSID Data Construction

The following SQL code reproduces the final PSID analysis dataset from the imported raw tables. The final exported table is FINAL\_DATASET\_COMBINET1\_LOGT1\_WORK5.

**Listing 1:** Final PSID Merge Code

```
1  /* final PSID merge */
2
3  /* PID maps */
4  CREATE TABLE MAP2007_HEAD AS
5  SELECT
6      ER30001 || '-' || ER30002 AS PID,
7      ER33901 AS FAMID_2007
8  FROM ID2007
9  WHERE ER33903 = 10
10     AND ER33902 = 1;
11
12  CREATE TABLE MAP2009_HEAD AS
13  SELECT
14      ER30001 || '-' || ER30002 AS PID,
15      ER34001 AS FAMID_2009
16  FROM ID2009
17  WHERE ER34003 = 10
18     AND ER34002 = 1;
19
20  CREATE TABLE MAP2011_HEAD AS
21  SELECT
22      ER30001 || '-' || ER30002 AS PID,
23      ER34101 AS FAMID_2011
24  FROM ID2011
25  WHERE ER34103 = 10
26     AND ER34102 = 1;
27
28  CREATE TABLE MAP2013_HEAD AS
29  SELECT
30      ER30001 || '-' || ER30002 AS PID,
31      ER34201 AS FAMID_2013
32  FROM ID2013
```

```

33 WHERE ER34203 = 10
34     AND ER34202 = 1;
35
36 CREATE TABLE MAP2015_HEAD AS
37 SELECT
38     ER30001 || '-' || ER30002 AS PID,
39     ER34301 AS FAMID_2015
40 FROM ID2015
41 WHERE ER34303 = 10
42     AND ER34302 = 1;
43
44 CREATE TABLE MAP2019_HEAD AS
45 SELECT
46     ER30001 || '-' || ER30002 AS PID,
47     ER34701 AS FAMID_2019
48 FROM ID2019
49 WHERE ER34703 = 10
50     AND ER34702 = 1;
51
52 CREATE TABLE MAP2021_HEAD AS
53 SELECT
54     ER30001 || '-' || ER30002 AS PID,
55     ER34901 AS FAMID_2021
56 FROM ID2021
57 WHERE ER34903 = 10
58     AND ER34902 = 1;
59
60
61 /* X to PID */
62 DECLARE
63     v_sql CLOB;
64 BEGIN
65     SELECT 'CREATE TABLE X2007_FULL_PID_WORK5 AS SELECT m.PID, ' ||
66         LISTAGG('x.' || column_name || ' AS ' || column_name || '_2007', ', ', ')
67         WITHIN GROUP (ORDER BY column_id) ||
68         ' FROM X2007_WORK5 x LEFT JOIN MAP2007_HEAD m ON x.ER36002 = m.FAMID_2007'
69     INTO v_sql
70 FROM user_tab_columns
71 WHERE table_name = 'X2007_WORK5'
72     AND column_name <> 'ER36002';
73

```

```

74 EXECUTE IMMEDIATE v_sql;
75 END;
76 /
77
78 DECLARE
79     v_sql CLOB;
80 BEGIN
81     SELECT 'CREATE TABLE X2009_FULL_PID_WORK5 AS SELECT m.PID, ' ||
82         LISTAGG('x.' || column_name || ' AS ' || column_name || '_2009', ', ', ')
83         WITHIN GROUP (ORDER BY column_id) ||
84         ' FROM X2009_WORK5 x LEFT JOIN MAP2009_HEAD m ON x.ER42002 = m.FAMID_2009'
85     INTO v_sql
86 FROM user_tab_columns
87 WHERE table_name = 'X2009_WORK5'
88     AND column_name <> 'ER42002';
89
90 EXECUTE IMMEDIATE v_sql;
91 END;
92 /
93
94 DECLARE
95     v_sql CLOB;
96 BEGIN
97     SELECT 'CREATE TABLE X2011_FULL_PID_WORK5 AS SELECT m.PID, ' ||
98         LISTAGG('x.' || column_name || ' AS ' || column_name || '_2011', ', ', ')
99         WITHIN GROUP (ORDER BY column_id) ||
100        ' FROM X2011_WORK4 x LEFT JOIN MAP2011_HEAD m ON x.ER47302 = m.FAMID_2011'
101    INTO v_sql
102 FROM user_tab_columns
103 WHERE table_name = 'X2011_WORK4'
104     AND column_name <> 'ER47302';
105
106 EXECUTE IMMEDIATE v_sql;
107 END;
108 /
109
110
111 /* T to PID */
112 DECLARE
113     v_sql CLOB;
114 BEGIN

```

```

115 SELECT 'CREATE TABLE T2013_PID AS SELECT m.PID, ' ||
116         LISTAGG('t.' || column_name || ' AS ' || column_name || '_2013', ', ', ')
117         WITHIN GROUP (ORDER BY column_id) ||
118         ' FROM T2013_RAW t LEFT JOIN MAP2013_HEAD m ON t.ER53002 = m.FAMID_2013'
119 INTO v_sql
120 FROM user_tab_columns
121 WHERE table_name = 'T2013_RAW'
122        AND column_name <> 'ER53002';
123
124 EXECUTE IMMEDIATE v_sql;
125 END;
126 /
127
128 DECLARE
129     v_sql CLOB;
130 BEGIN
131     SELECT 'CREATE TABLE T2015_PID AS SELECT m.PID, ' ||
132           LISTAGG('t.' || column_name || ' AS ' || column_name || '_2015', ', ', ')
133           WITHIN GROUP (ORDER BY column_id) ||
134           ' FROM T2015_RAW t LEFT JOIN MAP2015_HEAD m ON t.ER60002 = m.FAMID_2015'
135 INTO v_sql
136 FROM user_tab_columns
137 WHERE table_name = 'T2015_RAW'
138        AND column_name <> 'ER60002';
139
140 EXECUTE IMMEDIATE v_sql;
141 END;
142 /
143
144
145 /* Y to PID */
146 CREATE TABLE Y2019_PID AS
147 SELECT
148     m.PID,
149     y.ER77448 AS OUTCOME_2019
150 FROM Y_RAW y
151 LEFT JOIN MAP2019_HEAD m
152     ON y.ER72002 = m.FAMID_2019;
153
154 CREATE TABLE Y2021_PID AS
155 SELECT

```



```

156     m.PID,
157     y.ER81775 AS OUTCOME_2021
158 FROM Y_RAW y
159 LEFT JOIN MAP2021_HEAD m
160     ON y.ER78002 = m.FAMID_2021;
161
162
163 /* base */
164 DECLARE
165     v_t13 CLOB;
166     v_t15 CLOB;
167     v_sql CLOB;
168 BEGIN
169     SELECT LISTAGG('t13.' || column_name, ', ')
170         WITHIN GROUP (ORDER BY column_id)
171     INTO v_t13
172 FROM user_tab_columns
173 WHERE table_name = 'T2013_PID'
174     AND column_name <> 'PID';
175
176     SELECT LISTAGG('t15.' || column_name, ', ')
177         WITHIN GROUP (ORDER BY column_id)
178     INTO v_t15
179 FROM user_tab_columns
180 WHERE table_name = 'T2015_PID'
181     AND column_name <> 'PID';
182
183 v_sql :=
184     'CREATE TABLE BASE_TOTAL_KEEP AS
185     SELECT x11.PID' ||
186     CASE WHEN v_t13 IS NOT NULL THEN ', ' || v_t13 ELSE '' END ||
187     CASE WHEN v_t15 IS NOT NULL THEN ', ' || v_t15 ELSE '' END ||
188     ', y19.OUTCOME_2019,
189     y21.OUTCOME_2021
190 FROM X2011_FULL_PID_WORK5 x11
191 LEFT JOIN T2013_PID t13
192     ON x11.PID = t13.PID
193 LEFT JOIN T2015_PID t15
194     ON x11.PID = t15.PID
195 LEFT JOIN Y2019_PID y19
196     ON x11.PID = y19.PID

```

```

197         LEFT JOIN Y2021_PID y21
198             ON x11.PID = y21.PID';
199
200     EXECUTE IMMEDIATE v_sql;
201 END;
202 /
203
204
205 /* merge X */
206 DECLARE
207     v_x07 CLOB;
208     v_x09 CLOB;
209     v_x11 CLOB;
210     v_sql CLOB;
211 BEGIN
212     SELECT LISTAGG('x07.' || column_name, ', ')
213         WITHIN GROUP (ORDER BY column_id)
214     INTO v_x07
215     FROM user_tab_columns
216     WHERE table_name = 'X2007_FULL_PID_WORK5'
217         AND column_name <> 'PID';
218
219     SELECT LISTAGG('x09.' || column_name, ', ')
220         WITHIN GROUP (ORDER BY column_id)
221     INTO v_x09
222     FROM user_tab_columns
223     WHERE table_name = 'X2009_FULL_PID_WORK5'
224         AND column_name <> 'PID';
225
226     SELECT LISTAGG('x11.' || column_name, ', ')
227         WITHIN GROUP (ORDER BY column_id)
228     INTO v_x11
229     FROM user_tab_columns
230     WHERE table_name = 'X2011_FULL_PID_WORK5'
231         AND column_name <> 'PID';
232
233     v_sql :=
234         'CREATE TABLE FINAL_TOTAL_WORK5X AS
235         SELECT b.*' ||
236         CASE WHEN v_x07 IS NOT NULL THEN ', ' || v_x07 ELSE '' END ||
237         CASE WHEN v_x09 IS NOT NULL THEN ', ' || v_x09 ELSE '' END ||

```

```

238     CASE WHEN v_x11 IS NOT NULL THEN ', ' || v_x11 ELSE '' END ||
239     ' FROM BASE_TOTAL_KEEP b
240     LEFT JOIN X2007_FULL_PID_WORK5 x07
241         ON b.PID = x07.PID
242     LEFT JOIN X2009_FULL_PID_WORK5 x09
243         ON b.PID = x09.PID
244     LEFT JOIN X2011_FULL_PID_WORK5 x11
245         ON b.PID = x11.PID';
246
247     EXECUTE IMMEDIATE v_sql;
248 END;
249 /
250
251
252 /* final treatment */
253 CREATE TABLE FINAL_TOTAL_OUTCOMEOK_TREATSTRICT_WORK5 AS
254 SELECT
255     f.*,
256     CASE
257         WHEN
258             ER53404_2013 = 1
259             OR ER53667_2013 = 1
260             OR ER57583_2013 IN (1,3,6,7,8,9,10)
261             OR ER57589_2013 IN (1,3,6,7,8,9,10)
262             OR ER57693_2013 IN (1,3,6,7,8,9,10)
263             OR ER57699_2013 IN (1,3,6,7,8,9,10)
264             OR ER60419_2015 = 1
265             OR ER60682_2015 = 1
266             OR ER64714_2015 IN (1,3,6,7,8,9,10)
267             OR ER64720_2015 IN (1,3,6,7,8,9,10)
268             OR ER64853_2015 IN (1,3,6,7,8,9,10)
269             OR ER64859_2015 IN (1,3,6,7,8,9,10)
270         THEN 1 ELSE 0
271     END AS TREAT_STRICT
272 FROM FINAL_TOTAL_WORK5X f
273 WHERE
274     (OUTCOME_2019 IS NOT NULL AND OUTCOME_2019 <> 0)
275     OR (OUTCOME_2021 IS NOT NULL AND OUTCOME_2021 <> 0);
276
277
278 /* keep final treatment */

```

```

279 CREATE TABLE FINAL_DATASET_COMBINET1_WORK5 AS
280 SELECT *
281 FROM FINAL_TOTAL_OUTCOMEOK_TREATSTRICT_WORK5;
282
283 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER53404_2013;
284 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER53667_2013;
285 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER57583_2013;
286 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER57589_2013;
287 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER57693_2013;
288 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER57699_2013;
289
290 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER60419_2015;
291 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER60682_2015;
292 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER64714_2015;
293 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER64720_2015;
294 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER64853_2015;
295 ALTER TABLE FINAL_DATASET_COMBINET1_WORK5 DROP COLUMN ER64859_2015;
296
297
298 /* slog outcomes */
299 CREATE TABLE FINAL_DATASET_COMBINET1_LOGT1_WORK5 AS
300 SELECT
301     f.*,
302     CASE
303         WHEN OUTCOME_2019 IS NULL THEN NULL
304         WHEN OUTCOME_2019 = 0 THEN 0
305         WHEN OUTCOME_2019 > 0 THEN LN(OUTCOME_2019 + 1)
306         ELSE -LN(ABS(OUTCOME_2019) + 1)
307     END AS OUTCOME_2019_SLOG,
308     CASE
309         WHEN OUTCOME_2021 IS NULL THEN NULL
310         WHEN OUTCOME_2021 = 0 THEN 0
311         WHEN OUTCOME_2021 > 0 THEN LN(OUTCOME_2021 + 1)
312         ELSE -LN(ABS(OUTCOME_2021) + 1)
313     END AS OUTCOME_2021_SLOG
314 FROM FINAL_DATASET_COMBINET1_WORK5 f;
315
316
317 /* checks */
318 SELECT COUNT(*) AS n_rows, COUNT(DISTINCT PID) AS n_pid
319 FROM FINAL_DATASET_COMBINET1_LOGT1_WORK5;

```

```

320
321 SELECT
322     SUM(TREAT_STRICT) AS n_treated,
323     SUM(CASE WHEN TREAT_STRICT = 0 THEN 1 ELSE 0 END) AS n_control
324 FROM FINAL_DATASET_COMBINET1_LOGT1_WORK5;
325
326 SELECT COUNT(*) AS null_pid
327 FROM FINAL_DATASET_COMBINET1_LOGT1_WORK5
328 WHERE PID IS NULL;
329
330
331 /* export */
332 SELECT *
333 FROM FINAL_DATASET_COMBINET1_LOGT1_WORK5;

```

## Pseudo-Text Generation

The script below shows a representative pseudo-text generation step used to convert pre-treatment PSID variables into text inputs  $W_i$ . In this example, variables from waves 2007, 2009, and 2011 are encoded with fixed rules such as yes/no mapping, age binning, \$1,000 rounding, and explicit [MISSING] tokens. The same general template-based idea is used to create pseudo-text inputs for the downstream embedding pipelines.

**Listing 2:** Pseudo-text generation from pre-treatment PSID variables

```

1  """
2  PSID W Generation Pipeline (Schema v1.0)
3  Template-only deterministic pseudo-text generation from pre-treatment records.
4  """
5
6  import pandas as pd
7  import numpy as np
8
9  INPUT_FILE = "FINAL_DATASET_COMBINET1_LOGT1_WORK5.xlsx"
10 OUTPUT_FILE = "PSID_W_pipelineC_outputs_generated.xlsx"
11
12 MISSING_TOKEN = "[MISSING]"
13 SCHEMA_VERSION = "v1.0"

```

```

14 WAVES = [2007, 2009, 2011]
15
16
17 def is_missing(x):
18     if pd.isna(x):
19         return True
20     s = str(x).strip().lower()
21     return s in {"", "nan", "none", "missing", "999", "9999",
22                 "99999", "98", "99", "998", "9998",
23                 "dk", "na", "n/a"}
24
25
26 def round_money_1k(x):
27     if is_missing(x):
28         return MISSING_TOKEN
29     try:
30         x = float(x)
31     except Exception:
32         return MISSING_TOKEN
33     x = int(round(x / 1000.0) * 1000)
34     return "${:,.0f}".format(x)
35
36
37 def age_bin_5yr(age):
38     if is_missing(age):
39         return MISSING_TOKEN
40     age = int(age)
41     lower = (age // 5) * 5
42     return f"{lower}-{lower + 4}"
43
44
45 def yes_no_map(x):
46     if is_missing(x):
47         return MISSING_TOKEN
48     try:
49         x = int(float(x))
50     except Exception:
51         return MISSING_TOKEN
52     if x == 1:
53         return "yes"
54     if x == 5:

```

```

55     return "no"
56     return MISSING_TOKEN
57
58
59 def safe_str(x):
60     return MISSING_TOKEN if is_missing(x) else str(x)
61
62
63 COLUMN_MAP = {
64     2007: {"age": "ER36018", "sex": "ER36019", "educ": "ER36020",
65           "marital": "ER36021", "fam_size": "ER36022",
66           "kids": "ER36023", "house_value": "ER36024",
67           "mortgage": "ER36025", "working_now": "ER36026",
68           "business": "ER36027", "years_worked": "ER36028",
69           "head_income": "ER36029", "family_income": "ER36030",
70           "checking": "ER36031", "stocks": "ER36032",
71           "ira": "ER36033", "wic": "ER36034",
72           "heart": "ER36035", "hypertension": "ER36036",
73           "asthma": "ER36037", "diabetes": "ER36038",
74           "arthritis": "ER36039", "cancer": "ER36040"},
75     2009: {"age": "ER42018", "sex": "ER42019", "educ": "ER42020",
76           "marital": "ER42021", "fam_size": "ER42022",
77           "kids": "ER42023", "house_value": "ER42024",
78           "mortgage": "ER42025", "working_now": "ER42026",
79           "business": "ER42027", "years_worked": "ER42028",
80           "head_income": "ER42029", "family_income": "ER42030",
81           "checking": "ER42031", "stocks": "ER42032",
82           "ira": "ER42033", "wic": "ER42034",
83           "heart": "ER42035", "hypertension": "ER42036",
84           "asthma": "ER42037", "diabetes": "ER42038",
85           "arthritis": "ER42039", "cancer": "ER42040"},
86     2011: {"age": "ER47318", "sex": "ER47319", "educ": "ER47320",
87           "marital": "ER47321", "fam_size": "ER47322",
88           "kids": "ER47323", "house_value": "ER47324",
89           "mortgage": "ER47325", "working_now": "ER47326",
90           "business": "ER47327", "years_worked": "ER47328",
91           "head_income": "ER47329", "family_income": "ER47330",
92           "checking": "ER47331", "stocks": "ER47332",
93           "ira": "ER47333", "credit_card_debt": "ER47334",
94           "wic": "ER47335", "heart": "ER47336",
95           "hypertension": "ER47337", "asthma": "ER47338",

```

```

96         "diabetes": "ER47339", "arthritis": "ER47340",
97         "cancer": "ER47341"}
98     }
99
100
101 def build_health_summary(row, wave):
102     mapping = COLUMN_MAP[wave]
103     condition_vars = {
104         "heart": "heart disease",
105         "hypertension": "hypertension",
106         "asthma": "asthma",
107         "diabetes": "diabetes",
108         "arthritis": "arthritis",
109         "cancer": "cancer"
110     }
111
112     conditions = []
113     for key, label in condition_vars.items():
114         col = mapping.get(key)
115         if col in row and yes_no_map(row[col]) == "yes":
116             conditions.append(label)
117
118     return "none" if len(conditions) == 0 else "; ".join(conditions)
119
120
121 def build_facts(row, wave):
122     mapping = COLUMN_MAP[wave]
123     facts = []
124
125     if mapping.get("age") in row:
126         facts.append(f"Age of head (binned): {age_bin_5yr(row[mapping['age']])}.")
127     if mapping.get("sex") in row:
128         facts.append(f"Sex of head: {safe_str(row[mapping['sex']])}.")
129     if mapping.get("educ") in row:
130         facts.append(f"Completed education (head): {safe_str(row[mapping['educ']])} years.")
131     if mapping.get("marital") in row:
132         facts.append(f"Marital status: {safe_str(row[mapping['marital']])}.")
133     if mapping.get("fam_size") in row:
134         facts.append(f"Family unit size ({wave}): {safe_str(row[mapping['fam_size']])}.")
135     if mapping.get("kids") in row:
136         facts.append(f"Children present ({wave}): {safe_str(row[mapping['kids']])}.")

```



```

137
138     if mapping.get("house_value") in row:
139         facts.append(f"House value: {round_money_1k(row[mapping['house_value']])}.")
140     if mapping.get("mortgage") in row:
141         facts.append(f"Has mortgage: {yes_no_map(row[mapping['mortgage']])}.")
142
143     if mapping.get("working_now") in row:
144         facts.append(f"Working now: {yes_no_map(row[mapping['working_now']])}.")
145     if mapping.get("business") in row:
146         facts.append(f"Business ownership: {yes_no_map(row[mapping['business']])}.")
147     if mapping.get("years_worked") in row:
148         facts.append(f"Years worked since age 18: {safe_str(row[mapping['years_worked']])}.")
149
150     if mapping.get("head_income") in row:
151         facts.append(f"Head labor income: {round_money_1k(row[mapping['head_income']])}.")
152     if mapping.get("family_income") in row:
153         facts.append(f"Family money income: {round_money_1k(row[mapping['family_income']])}.")
154
155     if mapping.get("checking") in row:
156         facts.append(f"Checking/saving balance: {round_money_1k(row[mapping['checking']])}.")
157     if mapping.get("stocks") in row:
158         facts.append(f"Stocks value: {round_money_1k(row[mapping['stocks']])}.")
159     if mapping.get("ira") in row:
160         facts.append(f"IRA/annuity value: {round_money_1k(row[mapping['ira']])}.")
161     if mapping.get("credit_card_debt") in row:
162         facts.append(f"Credit card/store card debt: {round_money_1k(row[mapping['credit_card_debt']])}.")
163
164     if mapping.get("wic") in row:
165         facts.append(f"Received WIC benefits: {yes_no_map(row[mapping['wic']])}.")
166
167     facts.append(f"Chronic conditions summary: {build_health_summary(row, wave)}.")
168
169     return facts
170
171
172 def build_template(row, wave):
173     facts = build_facts(row, wave)
174     pid = row.get("PID", row.get("HHID", "[ID]"))
175     return (f"HHID: {pid}. Pre-treatment wave: {wave}. "
176           + " ".join(facts)
177           + f" schema: {SCHEMA_VERSION}")

```

```

178
179
180 def build_trajectory(row):
181     pieces = []
182     for wave in WAVES:
183         pieces.append(f"WAVE {wave}] {build_template(row, wave)}")
184     return " ".join(pieces)
185
186
187 def main():
188     df = pd.read_excel(INPUT_FILE)
189     outputs = []
190
191     for _, row in df.iterrows():
192         out = {}
193
194         if "PID" in df.columns:
195             out["PID"] = row["PID"]
196
197         if "TREAT_STRICT" in df.columns:
198             out["TREAT_STRICT"] = row["TREAT_STRICT"]
199
200         for wave in WAVES:
201             out[f"W_template_{wave}"] = build_template(row, wave)
202
203         out["W_traj_pipelineC_2007_2009_2011"] = build_trajectory(row)
204         outputs.append(out)
205
206     out_df = pd.DataFrame(outputs)
207     out_df.to_excel(OUTPUT_FILE, index=False)
208
209     print("Done.")
210     print(f"Saved to: {OUTPUT_FILE}")
211
212 if __name__ == "__main__":
213     main()

```

## Embedding Pipeline Code

This section presents the embedding pipeline code used to map pseudo-text  $W_i$  into low-dimensional representations  $R_i$  for causal adjustment.

**Listing 3:** Embedding pipeline reference implementation

```
1  #!/usr/bin/env python3
2  """
3  Part 2 embedding pipelines.
4
5  Pipeline A: TF-IDF -> TruncatedSVD
6  Pipeline B: SentenceTransformer -> PCA
7  Pipeline C: Trajectory concat -> Transformer -> PCA
8  Pipeline D: Multiview split -> Transformer -> concat -> PCA
9  """
10
11  from __future__ import annotations
12
13  import argparse
14  import json
15  import os
16  import re
17  from dataclasses import dataclass
18  from typing import Tuple
19
20  import joblib
21  import numpy as np
22  import pandas as pd
23  from sklearn.decomposition import PCA, TruncatedSVD
24  from sklearn.feature_extraction.text import TfidfVectorizer
25  from sklearn.model_selection import StratifiedKFold
26
27
28  def ensure_dir(path: str) -> None:
29      os.makedirs(path, exist_ok=True)
30
31
32  def stable_zscore(train, test):
33      mu = train.mean(axis=0)
34      sd = train.std(axis=0, ddof=0)
```

```

35     sd = np.where(sd == 0, 1.0, sd)
36     return (train - mu) / sd, (test - mu) / sd, mu, sd
37
38
39 def l2_normalize(E, eps=1e-12):
40     norms = np.linalg.norm(E, axis=1, keepdims=True)
41     return E / np.maximum(norms, eps)
42
43
44 def make_folds(df, id_col, treat_col, n_splits, seed):
45     if treat_col not in df.columns:
46         raise ValueError(f"treat_col '{treat_col}' not in dataframe.")
47
48     y = df[treat_col].astype(int).values
49     skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
50                           random_state=seed)
51
52     folds = np.zeros(len(df), dtype=int)
53
54     for fold_idx, (_, test_idx) in enumerate(skf.split(np.zeros(len(df)), y)):
55         folds[test_idx] = fold_idx
56
57     return pd.Series(folds, index=df.index, name="fold")
58
59
60 def normalize_text_series(s):
61     s = s.fillna("MISSING_NOTE").astype(str)
62     return s.str.replace(r"\s+", " ", regex=True).str.strip()
63
64
65 def safe_name(s):
66     return re.sub(r"[^A-Za-z0-9_.-]+", "_", s)
67
68
69 @dataclass
70 class TfidfConfig:
71     ngram_range: Tuple[int, int] = (1, 2)
72     min_df: int = 10
73     max_df: float = 0.90
74     max_features: int = 50000
75     sublinear_tf: bool = True

```

```

76     lowercase: bool = True
77
78
79 @dataclass
80 class TransformerConfig:
81     model_name: str = "sentence-transformers/all-MiniLM-L6-v2"
82     batch_size: int = 32
83     max_seq_length: int = 256
84
85
86 def run_pipeline_A_tfidf_svd(df, id_col, text_col, fold_col,
87                             out_dir, cache_dir, dims, tfidf_cfg):
88     ensure_dir(out_dir)
89     ensure_dir(cache_dir)
90
91     texts = normalize_text_series(df[text_col])
92     folds = df[fold_col].astype(int).values
93     K = int(folds.max()) + 1
94
95     for d in dims:
96         R = np.zeros((len(df), d), dtype=np.float32)
97
98         for k in range(K):
99             train_idx = np.where(folds != k)[0]
100             test_idx = np.where(folds == k)[0]
101
102             fold_cache = os.path.join(cache_dir, "A_tfidf_svd",
103                                       f"dim={d}", f"fold={k}")
104             ensure_dir(fold_cache)
105
106             testR_path = os.path.join(fold_cache, "R_test.npy")
107
108             if os.path.exists(testR_path):
109                 R[test_idx] = np.load(testR_path)
110                 continue
111
112             vectorizer = TfidfVectorizer(
113                 ngram_range=tfidf_cfg.ngram_range,
114                 min_df=tfidf_cfg.min_df,
115                 max_df=tfidf_cfg.max_df,
116                 max_features=tfidf_cfg.max_features,

```

```

117         sublinear_tf=tfidf_cfg.sublinear_tf,
118         lowercase=tfidf_cfg.lowercase,
119     )
120
121     V_train = vectorizer.fit_transform(texts.iloc[train_idx])
122     V_test = vectorizer.transform(texts.iloc[test_idx])
123
124     svd = TruncatedSVD(n_components=d, random_state=0)
125     R_train = svd.fit_transform(V_train).astype(np.float32)
126     R_test = svd.transform(V_test).astype(np.float32)
127
128     _, R_test_z, mu, sd = stable_zscore(R_train, R_test)
129     R_test_z = R_test_z.astype(np.float32)
130
131     joblib.dump(vectorizer, os.path.join(fold_cache, "tfidf.joblib"))
132     joblib.dump(svd, os.path.join(fold_cache, "svd.joblib"))
133     np.save(testR_path, R_test_z)
134     np.savez(os.path.join(fold_cache, "zscore_stats.npz"),
135             mu=mu, sd=sd)
136
137     R[test_idx] = R_test_z
138
139     out_path = os.path.join(out_dir, f"R_A_tfidf_svd_d{d}.csv")
140     out_df = pd.DataFrame(R, columns=[f"R{i+1}" for i in range(d)])
141     out_df.insert(0, fold_col, df[fold_col].values)
142     out_df.insert(0, id_col, df[id_col].values)
143     out_df.to_csv(out_path, index=False)
144
145     print(f"[A] wrote {out_path}")
146
147
148 def embed_texts_sentence_transformer(texts, cfg, cache_path):
149     if os.path.exists(cache_path):
150         return np.load(cache_path)
151
152     try:
153         from sentence_transformers import SentenceTransformer
154     except ImportError as e:
155         raise ImportError("Install with: pip install sentence-transformers") from e
156
157     model = SentenceTransformer(cfg.model_name)

```

```

158     model.max_seq_length = int(cfg.max_seq_length)
159
160     E = model.encode(
161         texts,
162         batch_size=int(cfg.batch_size),
163         show_progress_bar=True,
164         convert_to_numpy=True,
165         normalize_embeddings=False
166     ).astype(np.float32)
167
168     np.save(cache_path, E)
169     return E
170
171
172 def run_pipeline_B_transformer_pca(df, id_col, text_col, fold_col,
173                                   out_dir, cache_dir, ks, tr_cfg,
174                                   schema_v, prefix="B_singlewave"):
175     ensure_dir(out_dir)
176     ensure_dir(cache_dir)
177
178     texts = normalize_text_series(df[text_col]).tolist()
179     folds = df[fold_col].astype(int).values
180     K = int(folds.max()) + 1
181
182     base_dir = os.path.join(
183         cache_dir, "embeddings",
184         f"schema={schema_v}",
185         f"model={safe_name(tr_cfg.model_name)}",
186         f"maxlen={tr_cfg.max_seq_length}"
187     )
188     ensure_dir(base_dir)
189
190     E = embed_texts_sentence_transformer(
191         texts,
192         tr_cfg,
193         os.path.join(base_dir, f"{prefix}_E.npy")
194     )
195     E = l2_normalize(E)
196
197     for kdim in ks:
198         R = np.zeros((len(df), kdim), dtype=np.float32)

```

```

199
200     for kfold in range(K):
201         train_idx = np.where(folds != kfold)[0]
202         test_idx = np.where(folds == kfold)[0]
203
204         fold_cache = os.path.join(cache_dir, f"{prefix}_pca",
205                                   f"k={kdim}", f"fold={kfold}")
206         ensure_dir(fold_cache)
207
208         testR_path = os.path.join(fold_cache, "R_test.npy")
209
210         if os.path.exists(testR_path):
211             R[test_idx] = np.load(testR_path)
212             continue
213
214         pca = PCA(n_components=kdim,
215                  svd_solver="randomized",
216                  random_state=0)
217
218         R_train = pca.fit_transform(E[train_idx]).astype(np.float32)
219         R_test = pca.transform(E[test_idx]).astype(np.float32)
220
221         _, R_test_z, mu, sd = stable_zscore(R_train, R_test)
222         R_test_z = R_test_z.astype(np.float32)
223
224         joblib.dump(pca, os.path.join(fold_cache, "pca.joblib"))
225         np.save(testR_path, R_test_z)
226         np.savez(os.path.join(fold_cache, "zscore_stats.npz"),
227                 mu=mu, sd=sd)
228
229         R[test_idx] = R_test_z
230
231     out_path = os.path.join(out_dir, f"R_{prefix}_pca_k{kdim}.csv")
232     out_df = pd.DataFrame(R, columns=[f"R{i+1}" for i in range(kdim)])
233     out_df.insert(0, fold_col, df[fold_col].values)
234     out_df.insert(0, id_col, df[id_col].values)
235     out_df.to_csv(out_path, index=False)
236
237     print(f"[{prefix}] wrote {out_path}")
238
239

```



```

240 def build_trajectory_text_from_wide(df, traj_cols, wave_labels=None):
241     if wave_labels is None:
242         wave_labels = traj_cols
243
244     parts = []
245
246     for col, lab in zip(traj_cols, wave_labels):
247         if col not in df.columns:
248             raise ValueError(f"Trajectory column '{col}' not found.")
249
250         s = df[col].fillna("[MISSING_WAVE]").astype(str)
251         s = s.str.replace(r"\s+", " ", regex=True).str.strip()
252         parts.append("[WAVE " + str(lab) + "] " + s)
253
254     out = parts[0]
255
256     for p in parts[1:]:
257         out = out + "\n" + p
258
259     return out
260
261
262 def run_pipeline_C_trajectory(df, id_col, fold_col, out_dir,
263                             cache_dir, ks, tr_cfg, schema_v,
264                             traj_cols):
265     df = df.copy()
266
267     df["W_traj"] = build_trajectory_text_from_wide(
268         df,
269         traj_cols=traj_cols,
270         wave_labels=traj_cols
271     )
272
273     run_pipeline_B_transformer_pca(
274         df=df,
275         id_col=id_col,
276         text_col="W_traj",
277         fold_col=fold_col,
278         out_dir=out_dir,
279         cache_dir=cache_dir,
280         ks=ks,

```

```

281         tr_cfg=tr_cfg,
282         schema_v=schema_v,
283         prefix=f"C_traj_{'_'}.join(traj_cols)}"
284     )
285
286
287 TAG_RE = re.compile(r"\[[A-Z0-9_]+\]")
288
289
290 def split_sections_by_tags(note):
291     note = str(note)
292     matches = list(TAG_RE.finditer(note))
293
294     if not matches:
295         return {"ALL": note}
296
297     sections = {}
298
299     for i, m in enumerate(matches):
300         tag = m.group(1)
301         start = m.end()
302         end = matches[i + 1].start() if i + 1 < len(matches) else len(note)
303         sections[tag] = note[start:end].strip()
304
305     return sections
306
307
308 def build_multiview_texts(df, text_col):
309     W = normalize_text_series(df[text_col]).tolist()
310     work_texts = []
311     house_texts = []
312
313     for w in W:
314         sec = split_sections_by_tags(w)
315
316         if "ALL" in sec:
317             work_texts.append(w)
318             house_texts.append(w)
319         else:
320             work = " ".join([
321                 sec.get("EMP", ""),

```

```

322         sec.get("INCOME", ""),
323         sec.get("WORK", "")
324     ]).strip()
325
326     house = " ".join([
327         sec.get("DEMO", ""),
328         sec.get("HEALTH", ""),
329         sec.get("ASSET", ""),
330         sec.get("HOUSEHOLD", "")
331     ]).strip()
332
333     work_texts.append(work if work else w)
334     house_texts.append(house if house else w)
335
336     return work_texts, house_texts
337
338
339 def run_pipeline_D_multiview(df, id_col, text_col, fold_col,
340                             out_dir, cache_dir, ks, tr_cfg,
341                             schema_v):
342     ensure_dir(out_dir)
343     ensure_dir(cache_dir)
344
345     folds = df[fold_col].astype(int).values
346     work_texts, house_texts = build_multiview_texts(df, text_col)
347
348     base_dir = os.path.join(
349         cache_dir, "embeddings",
350         f"schema={schema_v}",
351         f"model={safe_name(tr_cfg.model_name)}",
352         f"maxlen={tr_cfg.max_seq_length}"
353     )
354     ensure_dir(base_dir)
355
356     Ew = embed_texts_sentence_transformer(
357         work_texts,
358         tr_cfg,
359         os.path.join(base_dir, "D_multiview_work_E.npy")
360     )
361
362     Eh = embed_texts_sentence_transformer(

```

```

363     house_texts,
364     tr_cfg,
365     os.path.join(base_dir, "D_multiview_house_E.npy")
366 )
367
368 Ew = l2_normalize(Ew)
369 Eh = l2_normalize(Eh)
370
371 Ecat = np.concatenate([Ew, Eh], axis=1).astype(np.float32)
372 K = int(folds.max()) + 1
373
374 for kdim in ks:
375     R = np.zeros((len(df), kdim), dtype=np.float32)
376
377     for kfold in range(K):
378         train_idx = np.where(folds != kfold)[0]
379         test_idx = np.where(folds == kfold)[0]
380
381         fold_cache = os.path.join(cache_dir, "D_multiview_pca",
382                                     f"k={kdim}", f"fold={kfold}")
383         ensure_dir(fold_cache)
384
385         testR_path = os.path.join(fold_cache, "R_test.npy")
386
387         if os.path.exists(testR_path):
388             R[test_idx] = np.load(testR_path)
389             continue
390
391         pca = PCA(n_components=kdim,
392                   svd_solver="randomized",
393                   random_state=0)
394
395         R_train = pca.fit_transform(Ecat[train_idx]).astype(np.float32)
396         R_test = pca.transform(Ecat[test_idx]).astype(np.float32)
397
398         _, R_test_z, mu, sd = stable_zscore(R_train, R_test)
399         R_test_z = R_test_z.astype(np.float32)
400
401         joblib.dump(pca, os.path.join(fold_cache, "pca.joblib"))
402         np.save(testR_path, R_test_z)
403         np.savez(os.path.join(fold_cache, "zscore_stats.npz"),

```

```

404         mu=mu, sd=sd)
405
406         R[test_idx] = R_test_z
407
408         out_path = os.path.join(out_dir, f"R_D_multiview_pca_k{kdim}.csv")
409         out_df = pd.DataFrame(R, columns=[f"R{i+1}" for i in range(kdim)])
410         out_df.insert(0, fold_col, df[fold_col].values)
411         out_df.insert(0, id_col, df[id_col].values)
412         out_df.to_csv(out_path, index=False)
413
414         print(f"[D] wrote {out_path}")
415
416
417 def main():
418     ap = argparse.ArgumentParser()
419
420     ap.add_argument("--data", required=True)
421     ap.add_argument("--out_dir", required=True)
422     ap.add_argument("--cache_dir", default="cache_part2")
423     ap.add_argument("--schema_v", default="schema_v1")
424
425     ap.add_argument("--id_col", required=True)
426     ap.add_argument("--text_col", required=True)
427     ap.add_argument("--treat_col", default=None)
428     ap.add_argument("--fold_col", default="fold")
429     ap.add_argument("--n_folds", type=int, default=5)
430     ap.add_argument("--seed", type=int, default=123)
431
432     ap.add_argument("--run_A", action="store_true")
433     ap.add_argument("--run_B", action="store_true")
434     ap.add_argument("--run_C", action="store_true")
435     ap.add_argument("--run_D", action="store_true")
436     ap.add_argument("--traj_cols", nargs="*", default=None)
437
438     ap.add_argument("--A_dims", nargs="*", type=int, default=[100, 300])
439     ap.add_argument("--B_dims", nargs="*", type=int, default=[50, 100, 200])
440     ap.add_argument("--C_dims", nargs="*", type=int, default=[50, 100])
441     ap.add_argument("--D_dims", nargs="*", type=int, default=[50, 100])
442
443     ap.add_argument("--min_df", type=int, default=10)
444     ap.add_argument("--max_df", type=float, default=0.90)

```

```

445     ap.add_argument("--max_features", type=int, default=50000)
446
447     ap.add_argument("--model_name",
448                     default="sentence-transformers/all-MiniLM-L6-v2")
449     ap.add_argument("--batch_size", type=int, default=32)
450     ap.add_argument("--max_len", type=int, default=256)
451
452     args = ap.parse_args()
453
454     df = pd.read_csv(args.data)
455
456     for col in [args.id_col, args.text_col]:
457         if col not in df.columns:
458             raise ValueError(f"Missing required column: {col}")
459
460     if args.fold_col not in df.columns:
461         if not args.treat_col:
462             raise ValueError("fold_col missing and --treat_col not provided.")
463
464     df[args.fold_col] = make_folds(
465         df,
466         args.id_col,
467         args.treat_col,
468         args.n_folds,
469         args.seed
470     )
471
472     ensure_dir(args.out_dir)
473     df[[args.id_col, args.fold_col]].to_csv(
474         os.path.join(args.out_dir, "folds.csv"),
475         index=False
476     )
477
478     ensure_dir(args.out_dir)
479     ensure_dir(args.cache_dir)
480
481     with open(os.path.join(args.out_dir, "run_config.json"),
482              "w", encoding="utf-8") as f:
483         json.dump(vars(args), f, indent=2)
484
485     tfidf_cfg = TfidfConfig(

```

```

486         min_df=args.min_df,
487         max_df=args.max_df,
488         max_features=args.max_features
489     )
490
491     tr_cfg = TransformerConfig(
492         model_name=args.model_name,
493         batch_size=args.batch_size,
494         max_seq_length=args.max_len
495     )
496
497     if args.run_A:
498         run_pipeline_A_tfidf_svd(
499             df=df,
500             id_col=args.id_col,
501             text_col=args.text_col,
502             fold_col=args.fold_col,
503             out_dir=args.out_dir,
504             cache_dir=args.cache_dir,
505             dims=args.A_dims,
506             tfidf_cfg=tfidf_cfg
507         )
508
509     if args.run_B:
510         run_pipeline_B_transformer_pca(
511             df=df,
512             id_col=args.id_col,
513             text_col=args.text_col,
514             fold_col=args.fold_col,
515             out_dir=args.out_dir,
516             cache_dir=args.cache_dir,
517             ks=args.B_dims,
518             tr_cfg=tr_cfg,
519             schema_v=args.schema_v,
520             prefix="B_singlewave"
521         )
522
523     if args.run_C:
524         if not args.traj_cols:
525             raise ValueError("--run_C requires --traj_cols.")
526

```

```

527     run_pipeline_C_trajectory(
528         df=df,
529         id_col=args.id_col,
530         fold_col=args.fold_col,
531         out_dir=args.out_dir,
532         cache_dir=args.cache_dir,
533         ks=args.C_dims,
534         tr_cfg=tr_cfg,
535         schema_v=args.schema_v,
536         traj_cols=args.traj_cols
537     )
538
539     if args.run_D:
540         run_pipeline_D_multiview(
541             df=df,
542             id_col=args.id_col,
543             text_col=args.text_col,
544             fold_col=args.fold_col,
545             out_dir=args.out_dir,
546             cache_dir=args.cache_dir,
547             ks=args.D_dims,
548             tr_cfg=tr_cfg,
549             schema_v=args.schema_v
550         )
551
552     if not (args.run_A or args.run_B or args.run_C or args.run_D):
553         print("Nothing to run. Pass at least one of --run_A --run_B --run_C --run_D")
554
555
556 if __name__ == "__main__":
557     main()

```



## MC Study

The script below shows the MC study conducted across the pipelines and semi-synthetic datasets.

**Listing 4:** Analysis code for MC study

```
1  """
2  mc_experiment_pipeline.py
3  =====
4  Monte Carlo pipeline for evaluating AIPW causal-effect estimators under three
5  feature conditions (X-only, R-only, X+R) across three CATE scenarios
6  (Constant, Linear, Nonlinear).
7
8  Stages
9  -----
10 1. DGP: semi-synthetic panel simulation
11 2. AIPW: cross-fit doubly-robust estimation (Ridge / Lasso / RF / NN)
12 3. MC orchestration: M independent seeds
13 4. Aggregation: long-form CSVs and summary tables
14
15 Configuration
16 -----
17 Edit the CONFIG block to match your dataset and directory layout.
18 """
19
20 import copy
21 import json
22 import os
23 import time
24 import warnings
25
26 import numpy as np
27 import pandas as pd
28 import torch
29 import torch.nn as nn
30 import torch.optim as optim
31
32 from sklearn.base import BaseEstimator, RegressorMixin
33 from sklearn.decomposition import PCA
34 from sklearn.ensemble import RandomForestRegressor
```

```

35 from sklearn.linear_model import (
36     LassoCV, LinearRegression, LogisticRegression, RidgeCV,
37 )
38 from sklearn.metrics import accuracy_score, mean_squared_error, r2_score
39 from sklearn.model_selection import KFold, train_test_split
40 from sklearn.preprocessing import StandardScaler
41
42 warnings.filterwarnings("ignore")
43
44
45 # =====
46 # CONFIG: edit these to match your dataset
47 # =====
48
49 DATA_FILE = "panel_data.xlsx"
50 SHEET_NAME = "Data"
51
52 # Pre-treatment income columns used to build the history matrix H
53 INCOME_BASE_COL = "INCOME_T1"
54 INCOME_LAGGED_COL = "INCOME_T0"
55 Y_BASE_COL = INCOME_BASE_COL
56
57 # Suffixes that identify pre-treatment covariate columns
58 PRETREATMENT_SUFFIXES = ("_T0", "_T1")
59
60 # Columns that must never be treated as covariates
61 NON_FEATURE_COLS = {
62     "PID", "fold", "Unnamed: 0",
63     "TREAT_STRICT",
64     "OUTCOME_2019", "OUTCOME_2021",
65     "OUTCOME_2019_SLOG", "OUTCOME_2021_SLOG",
66 }
67
68 # Treatment and outcome column names in the simulated CSVs
69 T_COL = "TREAT_STRICT"
70 Y_COL = "OUTCOME_AVG"
71 TRUE_ATE = 0.5
72
73 # DGP hyperparameters
74 SIGMA_Y, GAMMA_U, RHO_BASE, TAU0 = 1.0, 1.0, 0.5, 0.5
75 SIGMA_U_DEFAULT = 0.5

```

```

76 S_X_DEFAULT      = 0.2
77 S_U_DEFAULT      = 0.5
78 ALPHA_U_DEFAULT  = 1.0
79 EPS_CLIP         = 0.02
80 TARGET_T_RATE    = 0.20
81
82 # I/O
83 OUTPUT_DIR        = os.environ.get("X_DGP_DIR", "output")
84 MASTER_RECORD_FILE = "master_mc_records.json"
85 SUMMARY_DIR       = "mc_summary"
86 MC_M              = int(os.environ.get("MC_M", "5"))
87
88 SCENARIOS = {
89     "Constant": "simulation_tier3_constant.csv",
90     "Linear":   "simulation_tier3_linear.csv",
91     "Nonlinear": "simulation_tier3_nonlinear.csv",
92 }
93
94
95 # =====
96 # MODULE 1 Data generating process (DGP)
97 # =====
98
99 def signed_log(x):
100     x = np.asarray(x, dtype=float)
101     return np.sign(x) * np.log1p(np.abs(x))
102
103
104 def _zscore(a):
105     a = np.asarray(a, dtype=float)
106     s = a.std(ddof=0)
107     return (a - a.mean()) / (s if s > 0 else 1.0)
108
109
110 def _seed_rng():
111     env = os.environ.get("MC_SEED")
112     seed = int(env) if env is not None else 42
113     return np.random.default_rng(seed), seed
114
115
116 def get_x_tilde_cols(df):

```

```

117     """Return pre-treatment numeric covariate columns."""
118     return [
119         c for c in df.columns
120         if any(c.endswith(s) for s in PRETREATMENT_SUFFIXES)
121         and c not in NON_FEATURE_COLS
122         and np.issubdtype(df[c].dtype, np.number)
123     ]
124
125
126 def _detect_income_cols(df, x_cols):
127     return [c for c in x_cols
128             if len(df[c].dropna()) and np.nanmax(np.abs(df[c].dropna().values)) > 1000]
129
130
131 def build_x_tilde_matrix(df, x_cols):
132     income_cols = set(_detect_income_cols(df, x_cols))
133     out = np.empty((len(df), len(x_cols)), dtype=float)
134     for j, c in enumerate(x_cols):
135         v = df[c].fillna(df[c].median()).to_numpy(dtype=float)
136         if c in income_cols:
137             v = signed_log(v)
138         out[:, j] = _zscore(v)
139     return out, income_cols
140
141
142 def build_history_matrix(df):
143     """H = [income_mean, income_slope, income_vol, PCA_residual]."""
144     inc_base = signed_log(df[INCOME_BASE_COL].fillna(df[INCOME_BASE_COL].median()).to_numpy())
145     inc_lagged = signed_log(df[INCOME_LAGGED_COL].fillna(df[INCOME_LAGGED_COL].median()).to_numpy())
146     h_mean = (inc_base + inc_lagged) / 2.0
147     h_slope = inc_base - inc_lagged
148
149     x_cols = get_x_tilde_cols(df)
150     rem_cols = [c for c in x_cols if c not in {INCOME_BASE_COL, INCOME_LAGGED_COL}]
151     h_pca = (PCA(n_components=1).fit_transform(build_x_tilde_matrix(df, rem_cols)[0]).ravel()
152             if rem_cols else np.zeros(len(df)))
153
154     return np.column_stack([_zscore(h_mean), _zscore(h_slope),
155                             _zscore(np.abs(h_slope)), _zscore(h_pca)])
156
157

```

```

158 def build_latent_u(H, sigma_u=SIGMA_U_DEFAULT, rng=None):
159     if rng is None:
160         rng, _ = _seed_rng()
161     w = PCA(n_components=1).fit(H).components_[0]
162     z = _zscore(H @ w)
163     return z + sigma_u * rng.standard_normal(len(z)), w
164
165
166 def calibration_u_r2(U, X_tilde):
167     """Out-of-sample R^2 of linear U ~ X_tilde (5-fold CV)."""
168     if X_tilde.shape[1] == 0:
169         return float("nan")
170     kf, preds = KFold(n_splits=5, shuffle=True, random_state=0), np.zeros_like(U)
171     for tr, te in kf.split(X_tilde):
172         preds[te] = LinearRegression().fit(X_tilde[tr], U[tr]).predict(X_tilde[te])
173     ss_res = np.sum((U - preds) ** 2)
174     ss_tot = np.sum((U - U.mean()) ** 2)
175     return float(1.0 - ss_res / ss_tot) if ss_tot > 0 else 0.0
176
177
178 def _calibrate_alpha_0(linpred, target=TARGET_T_RATE, eps=EPS_CLIP,
179                        lo=-10.0, hi=10.0, tol=1e-4, n_iter=60):
180     """Binary-search intercept so that mean(clipped propensity) = target."""
181     for _ in range(n_iter):
182         mid = 0.5 * (lo + hi)
183         p = np.clip(1.0 / (1.0 + np.exp(-(mid + linpred))), eps, 1 - eps)
184         if abs(p.mean() - target) < tol:
185             return mid
186         lo, hi = (mid, hi) if p.mean() < target else (lo, mid)
187     return 0.5 * (lo + hi)
188
189
190 def assign_treatment(X_tilde, U, rng, s_x=S_X_DEFAULT, s_u=S_U_DEFAULT,
191                     alpha_u=ALPHA_U_DEFAULT, eps=EPS_CLIP, target=TARGET_T_RATE):
192     alpha_x = np.ones(X_tilde.shape[1]) / X_tilde.shape[1]
193     linpred = s_x * (X_tilde @ alpha_x) + s_u * alpha_u * U
194     alpha_0 = _calibrate_alpha_0(linpred, target=target, eps=eps)
195     p_raw = 1.0 / (1.0 + np.exp(-(alpha_0 + linpred)))
196     p = np.clip(p_raw, eps, 1 - eps)
197     T = (rng.random(len(p)) < p).astype(int)
198     return T, p, alpha_0, float(((p_raw < eps) | (p_raw > 1 - eps)).mean())

```

```

199
200
201 def _smd(X, T):
202     out = []
203     for j in range(X.shape[1]):
204         m1, m0 = X[T == 1, j].mean(), X[T == 0, j].mean()
205         denom = np.sqrt((X[T == 1, j].var() + X[T == 0, j].var()) / 2.0)
206         if denom > 0:
207             out.append(abs(m1 - m0) / denom)
208     return {"mean": float(np.mean(out)), "max": float(np.max(out))} if out else {"mean": 0.0, "max": 0.0}
209
210
211 def _save_scenario(df, output_dir, fname, U, g, tau, ps, T, Y0, Y1, Y_obs):
212     out = df.copy()
213     out.update({"true_u": U, "true_g": g, "true_tau": tau, "ps": ps,
214               "TREAT_STRICT": T, "Y0": Y0, "Y1": Y1, "OUTCOME_AVG": Y_obs})
215     out.to_csv(os.path.join(output_dir, fname), index=False)
216
217
218 def run_simulation(df, output_dir, do_plot=False):
219     """Generate constant / linear / nonlinear simulation CSVs + calibration dict."""
220     os.makedirs(output_dir, exist_ok=True)
221     rng, seed = _seed_rng()
222
223     x_cols = get_x_tilde_cols(df)
224     X_tilde, income_cols = build_x_tilde_matrix(df, x_cols)
225     H = build_history_matrix(df)
226     U, _ = build_latent_u(H, rng=rng)
227
228     Y_base = signed_log(df[Y_BASE_COL].fillna(df[Y_BASE_COL].median()).to_numpy())
229     p_dim = X_tilde.shape[1]
230     g = X_tilde @ np.full(p_dim, 0.1) + 1.5 * (X_tilde[:, 0] < 0) + np.maximum(0, X_tilde[:, 1] - 1)
231
232     def noise(): return rng.standard_normal(len(df)) * SIGMA_Y
233     def Y0_base(): return g + GAMMA_U * U + RHO_BASE * Y_base + noise()
234
235     # Constant CATE
236     T_c, ps_c, a0, clip = assign_treatment(X_tilde, U, rng)
237     Y0_c = Y0_base(); Y1_c = Y0_c + TAU0; Y_c = T_c * Y1_c + (1 - T_c) * Y0_c
238     _save_scenario(df, output_dir, "simulation_tier3_constant.csv",
239                   U, g, np.full(len(df), TAU0), ps_c, T_c, Y0_c, Y1_c, Y_c)

```

```

240
241 # Linear CATE
242 T_l, ps_l, _, _ = assign_treatment(X_tilde, U, rng)
243 tau_lin = TAU0 + X_tilde @ np.full(p_dim, 0.1) + 0.5 * U
244 Y0_l = Y0_base(); Y1_l = Y0_l + tau_lin; Y_l = T_l * Y1_l + (1 - T_l) * Y0_l
245 _save_scenario(df, output_dir, "simulation_tier3_linear.csv",
246               U, g, tau_lin, ps_l, T_l, Y0_l, Y1_l, Y_l)
247
248 # Nonlinear CATE (re-centred to E[tau] = TAU0)
249 T_n, ps_n, _, _ = assign_treatment(X_tilde, U, rng)
250 _het = np.tanh(X_tilde[:, 0]) + 0.8 * (X_tilde[:, 1] > 0) + 0.3 * U ** 2
251 tau_non = TAU0 + (_het - _het.mean())
252 Y0_n = Y0_base(); Y1_n = Y0_n + tau_non; Y_n = T_n * Y1_n + (1 - T_n) * Y0_n
253 _save_scenario(df, output_dir, "simulation_tier3_nonlinear.csv",
254               U, g, tau_non, ps_n, T_n, Y0_n, Y1_n, Y_n)
255
256 pct = np.percentile(ps_c, [1, 5, 50, 95, 99])
257 return {
258     "mc_seed": seed,
259     "n": int(len(df)),
260     "U_R2_on_X_tilde": calibration_u_r2(U, X_tilde),
261     "treatment_rate": float(T_c.mean()),
262     "ps_percentiles": dict(zip(["1", "5", "50", "95", "99"], pct.tolist())),
263     "fraction_clipped": float(clip),
264     "corr_U_T": float(np.corrcoef(U, T_c)[0, 1]),
265     "smd_X_tilde": _smd(X_tilde, T_c),
266     "alpha_0": float(a0),
267     "n_x_tilde_cols": int(p_dim),
268     "n_income_cols_logged": int(len(income_cols)),
269 }
270
271
272 # =====
273 # MODULE 2 Causal estimation (cross-fit AIPW)
274 # =====
275
276 class PyTorchMLPRegressor(BaseEstimator, RegressorMixin):
277     """MLP with internal Y-standardisation and early stopping."""
278
279     def __init__(self, hidden_layers=(32, 16), max_iter=300,
280                 patience=15, val_frac=0.15, random_state=42):

```

```

281     self.hidden_layers = hidden_layers
282     self.max_iter, self.patience = max_iter, patience
283     self.val_frac, self.random_state = val_frac, random_state
284     self.device = torch.device("cpu")
285
286     def _build(self, n_in):
287         layers, last = [], n_in
288         for d in self.hidden_layers:
289             layers += [nn.Linear(last, d), nn.ReLU()]; last = d
290         layers.append(nn.Linear(last, 1))
291         return nn.Sequential(*layers).to(self.device)
292
293     def fit(self, X, y):
294         torch.manual_seed(self.random_state)
295         y = np.asarray(y, dtype=np.float32)
296         self.y_mean_ = float(y.mean())
297         self.y_std_ = float(y.std()) or 1.0
298         y_z = (y - self.y_mean_) / self.y_std_
299
300         idx = np.random.default_rng(self.random_state).permutation(len(X))
301         n_val = max(1, int(len(X) * self.val_frac))
302         vi, ti = idx[:n_val], idx[n_val:]
303
304         t = lambda a: torch.tensor(a, dtype=torch.float32).to(self.device)
305         X_tr, y_tr = t(X[ti]), t(y_z[ti]).view(-1, 1)
306         X_va, y_va = t(X[vi]), t(y_z[vi]).view(-1, 1)
307
308         self.model = self._build(X.shape[1])
309         opt = torch.optim.AdamW(self.model.parameters(), lr=1e-3, weight_decay=1e-3)
310         crit = nn.MSELoss()
311         best_val, best_state, bad = float("inf"), None, 0
312
313         for _ in range(self.max_iter):
314             self.model.train(); opt.zero_grad()
315             crit(self.model(X_tr), y_tr).backward(); opt.step()
316             self.model.eval()
317             with torch.no_grad():
318                 v = crit(self.model(X_va), y_va).item()
319             if v < best_val - 1e-4:
320                 best_val, best_state, bad = v, copy.deepcopy(self.model.state_dict()), 0
321             else:

```



```

322         bad += 1
323         if bad >= self.patience:
324             break
325     if best_state:
326         self.model.load_state_dict(best_state)
327     return self
328
329     def predict(self, X):
330         self.model.eval()
331         with torch.no_grad():
332             z = self.model(torch.tensor(X, dtype=torch.float32).to(self.device)).cpu().numpy().flatten()
333         return z * self.y_std_ + self.y_mean_
334
335
336     def get_base_model(model_type):
337         return {
338             "Ridge": RidgeCV(alphas=np.logspace(-3, 3, 7)),
339             "Lasso": LassoCV(n_alphas=10, random_state=42, precompute=False, n_jobs=1),
340             "RF": RandomForestRegressor(n_estimators=30, max_depth=5, random_state=42, n_jobs=1),
341             "NN": PyTorchMLPRegressor(hidden_layers=(32, 16), max_iter=200, random_state=42),
342         }.get(model_type, LinearRegression())
343
344
345     def downsample_and_clean(df, X_cols, target_n=1000):
346         """Drop rows missing T/Y, stratified-downsample, mean-impute covariates."""
347         df = df.dropna(subset=[T_COL, Y_COL]).copy()
348         if len(df) > target_n:
349             _, df = train_test_split(df, test_size=target_n, stratify=df[T_COL], random_state=42)
350         keep = []
351         for col in X_cols:
352             if col not in df.columns or df[col].isna().all():
353                 continue
354             df[col] = df[col].fillna(df[col].mean())
355             keep.append(col)
356         X_cols[:] = keep
357         return df
358
359
360     def run_aipw_cross_fit(df, X_scaled, model_type, n_splits=3):
361         """Return per-observation doubly-robust influence scores."""
362         kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)

```

```

363     scores = np.zeros(len(df))
364     y, t     = df[Y_COL].values, df[T_COL].values
365     for tr, te in kf.split(df):
366         X_tr, X_ts = X_scaled[tr], X_scaled[te]
367         y_tr, y_ts = y[tr], y[te]
368         t_tr, t_ts = t[tr], t[te]
369         e = LogisticRegression(max_iter=1000).fit(X_tr, t_tr).predict_proba(X_ts)[: , 1].clip(0.01, 0.99)
370         m1 = get_base_model(model_type).fit(X_tr[t_tr == 1], y_tr[t_tr == 1])
371         m0 = get_base_model(model_type).fit(X_tr[t_tr == 0], y_tr[t_tr == 0])
372         dr1 = m1.predict(X_ts) + t_ts * (y_ts - m1.predict(X_ts)) / e
373         dr0 = m0.predict(X_ts) + (1 - t_ts) * (y_ts - m0.predict(X_ts)) / (1 - e)
374         scores[te] = dr1 - dr0
375     return scores
376
377
378 def _bootstrap_ci(scores, n_boot=500):
379     np.random.seed(42)
380     boots = np.random.choice(scores, size=(n_boot, len(scores)), replace=True).mean(axis=1)
381     ate    = float(scores.mean())
382     return ate, float(np.percentile(boots, 2.5)), float(np.percentile(boots, 97.5))
383
384
385 def estimate_causal_effects(df, X_cols, scenario_name):
386     """Return (estimates_dict, diagnostics_dict)."""
387     scaler    = StandardScaler()
388     X_scaled  = scaler.fit_transform(df[X_cols])
389     y, t      = df[Y_COL].values, df[T_COL].values
390
391     estimates = {}
392     for m_type in ["Ridge", "Lasso", "RF", "NN"]:
393         try:
394             est, lo, hi = _bootstrap_ci(run_aipw_cross_fit(df.copy(), X_scaled, m_type))
395             estimates[f"AIPW ({m_type})"] = {"Estimate": est, "CI_Lower": lo, "CI_Upper": hi}
396         except Exception:
397             pass
398
399     metrics = {}
400     X_tr_ps, X_ts_ps, t_tr, t_ts = train_test_split(X_scaled, t, test_size=0.2, random_state=42)
401     metrics["PS_Accuracy"] = float(
402         accuracy_score(t_ts, LogisticRegression(max_iter=1000).fit(X_tr_ps, t_tr).predict(X_ts_ps))
403     )

```

```

404 X_tr, X_ts, y_tr, y_ts = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
405 for m_label in ["Ridge", "Lasso", "RF", "NN"]:
406     y_pred = get_base_model(m_label).fit(X_tr, y_tr).predict(X_ts)
407     metrics[f"{m_label}_R2_OOS"] = float(r2_score(y_ts, y_pred))
408     metrics[f"{m_label}_MSE_OOS"] = float(mean_squared_error(y_ts, y_pred))
409
410     return estimates, metrics
411
412
413 # =====
414 # MODULE 3 Monte Carlo orchestrator
415 # =====
416
417 def run_mc_experiments(m=MC_M, output_dir=OUTPUT_DIR,
418                       master_record_file=MASTER_RECORD_FILE):
419     """Run M independent MC iterations, each with a fresh DGP seed.
420
421     Per iteration:
422     1. run_simulation() generates new semi-synthetic CSVs.
423     2. estimate_causal_effects() runs AIPW for each scenario.
424     3. The run record is appended to master_mc_records.json.
425     """
426     print(f"=== MC Experiment (M={m}) ===")
427     if os.path.exists(master_record_file):
428         os.remove(master_record_file)
429
430     base_df = pd.read_excel(DATA_FILE, sheet_name=SHEET_NAME)
431     master = {"runs": []}
432     start = time.time()
433
434     for i in range(1, m + 1):
435         print(f"\n=== Run {i}/{m} ===")
436         os.environ["MC_SEED"] = str(i * 1337)
437         os.environ["X_DGP_DIR"] = output_dir
438         os.makedirs(output_dir, exist_ok=True)
439
440         calibration = run_simulation(base_df, output_dir=output_dir)
441         print(f"  treatment_rate={calibration['treatment_rate']:.3f}  "
442               f"U_R2={calibration['U_R2_on_X_tilde']:.3f}  "
443               f"clip_frac={calibration['fraction_clipped']:.3f}")
444

```

```

445     records = []
446     for scenario, fname in SCENARIOS.items():
447         df = pd.read_csv(os.path.join(output_dir, fname))
448         x_cols = get_x_tilde_cols(df)
449         df = downsample_and_clean(df, x_cols)
450         res, met = estimate_causal_effects(df, x_cols, scenario)
451         records.append({
452             "Condition": "X_only",
453             "Pipeline": "X_baseline",
454             "Scenario": scenario,
455             "Estimates": res,
456             "Diagnostics": met,
457         })
458
459     master["runs"].append({"Run_ID": i, "Calibration": calibration, "Records": records})
460     with open(master_record_file, "w") as f:
461         json.dump(master, f, indent=2)
462
463     eta = (time.time() - start) / i * (m - i)
464     print(f" {len(records)} scenario records | ETA {eta/60:.1f}m")
465
466     print(f"\n=== Done: {master_record_file} ===")
467
468
469 # =====
470 # MODULE 4 Result aggregation
471 =====
472
473 def long_estimates(master):
474     rows = []
475     for run in master["runs"]:
476         for rec in run["Records"]:
477             for est_name, vals in rec.get("Estimates", {}).items():
478                 rows.append({
479                     "Run_ID": run["Run_ID"],
480                     "Condition": rec["Condition"],
481                     "Pipeline": rec["Pipeline"],
482                     "Scenario": rec["Scenario"],
483                     "Estimator": est_name,
484                     "Estimate": vals.get("Estimate"),
485                     "CI_Lower": vals.get("CI_Lower"),

```

```

486         "CI_Upper": vals.get("CI_Upper"),
487         "PS_Acc": rec.get("Diagnostics", {}).get("PS_Accuracy"),
488     })
489
490 df = pd.DataFrame(rows)
491 df["Bias"] = df["Estimate"] - TRUE_ATE
492 df["AbsBias"] = df["Bias"].abs()
493 df["SqError"] = df["Bias"] ** 2
494 df["CI_Width"] = df["CI_Upper"] - df["CI_Lower"]
495 df["Covers"] = ((df["CI_Lower"] <= TRUE_ATE) & (df["CI_Upper"] >= TRUE_ATE)).astype(int)
496
497 return df
498
499 def summarize_results(master_record_file=MASTER_RECORD_FILE, out_dir=SUMMARY_DIR):
500     os.makedirs(out_dir, exist_ok=True)
501     with open(master_record_file) as f:
502         master = json.load(f)
503
504     long_df = long_estimates(master)
505     long_df.to_csv(os.path.join(out_dir, "mc_estimates_long.csv"), index=False)
506
507     g = long_df.groupby(["Condition", "Pipeline", "Scenario", "Estimator"])
508
509     bias = g["AbsBias"].agg(["mean", "median", "std",
510                             lambda s: s.quantile(0.025),
511                             lambda s: s.quantile(0.975), "count"]).reset_index()
512     bias.columns = ["Condition", "Pipeline", "Scenario", "Estimator", "mean", "median", "sd", "q025", "q975", "n_runs"]
513     bias.to_csv(os.path.join(out_dir, "mc_bias_summary.csv"), index=False)
514
515     (g["Covers"].mean().reset_index().rename(columns={"Covers": "coverage"}))
516     .to_csv(os.path.join(out_dir, "mc_coverage_summary.csv"), index=False)
517
518     rmse = g["SqError"].mean().reset_index()
519     rmse["RMSE"] = np.sqrt(rmse["SqError"])
520     rmse[["Condition", "Pipeline", "Scenario", "Estimator", "RMSE"]].to_csv(
521         os.path.join(out_dir, "mc_rmse_summary.csv"), index=False)
522
523     # Calibration
524     cal_rows = []
525     for run in master["runs"]:
526         cal = run.get("Calibration") or {}
527         flat = {"Run_ID": run["Run_ID"]}

```

```

527         for k, v in cal.items():
528             if isinstance(v, dict):
529                 flat.update({f"{k}.{kk}": vv for kk, vv in v.items()})
530             else:
531                 flat[k] = v
532         cal_rows.append(flat)
533 cal_df = pd.DataFrame(cal_rows)
534 cal_df.to_csv(os.path.join(out_dir, "mc_calibration_long.csv"), index=False)
535 if not cal_df.empty:
536     (cal_df.drop(columns=["Run_ID"], errors="ignore").select_dtypes("number")
537      .agg(["mean", "std", "min", "max"]).T.reset_index()
538      .rename(columns={"index": "metric"})
539      .to_csv(os.path.join(out_dir, "mc_calibration_summary.csv"), index=False))
540
541 if __name__ == "__main__":
542     import argparse
543     parser = argparse.ArgumentParser()
544     parser.add_argument("--stage", choices=["mc", "summarize", "all"], default="all")
545     args = parser.parse_args()
546
547     os.makedirs(OUTPUT_DIR, exist_ok=True)
548
549     if args.stage in ("mc", "all"):
550         run_mc_experiments()
551     if args.stage in ("summarize", "all"):
552         summarize_results()

```